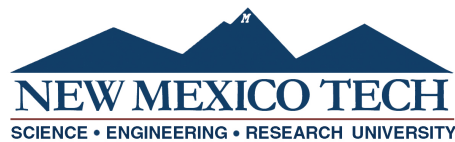


DESIGN AND EVALUATION OF AN EDUCATIONAL GAME TO TEACH FILE SYSTEM MANAGEMENT

by

Wesley Camphouse

Submitted in Partial Fulfillment
of the Requirements for the Degree of
Master of Science in Computer Science



New Mexico Institute of Mining and Technology
Socorro, New Mexico
May, 2025

This thesis is dedicated to my friends and family, without who I never would have completed this.

This research is to be presented at the 2025 i-ETC conference at Utah Valley University.

Wesley Camphouse
New Mexico Institute of Mining and Technology
May, 2025

ACKNOWLEDGMENTS

This thesis was typeset with $\text{\LaTeX}$ ¹ by the author.

¹The $\text{\LaTeX}$ document preparation system was developed by Leslie Lamport as a special version of Donald Knuth's $\text{\TeX}$ program for computer typesetting. $\text{\TeX}$ is a trademark of the American Mathematical Society. The $\text{\LaTeX}$ macro package for the New Mexico Institute of Mining and Technology thesis format was written by John W. Shipman.

ABSTRACT

Due to the prevalence of computers in modern society, the skills required to make effective use of a computer are very important. Despite this, research indicates that the computer and digital literacy of the general population may be lacking. To address this, an educational game was developed to teach late middle and early high school students about the proper organization of the digital file system. This game teaches students how to organize the digital file system by having them complete various puzzles requiring the navigation and use of the real file system of their machine. To better suit its purpose, the game also implements adaptive difficulty and automatic item generation to provide players with a variety of levels that maintain an appropriate challenge. Once designed this system was evaluated on middle and high school students enrolled in two New Mexico schools. The results collected indicate that the game was effective in teaching about the digital file system, despite certain drawbacks.

Keywords: Game-based learning; Adaptive learning; Automatic item generation; Computer literacy; Digital literacy

CONTENTS

	Page
LIST OF TABLES	vii
LIST OF FIGURES	viii
CHAPTER 1. INTRODUCTION	1
CHAPTER 2. LITERATURE REVIEW	3
2.1 Computer Literacy	3
2.2 Gamification and GBL	5
2.3 Adaptive Learning	8
2.4 Automatic Item Generation	9
2.5 Similar Systems	11
2.6 Research Questions	13
CHAPTER 3. RESEARCH METHODS	14
3.1 Game-Based Learning	14
3.2 Adaptive Difficulty	16
3.3 Automatic Item Generation	20
3.4 Summary	21
CHAPTER 4. SYSTEM	23
4.1 Tutorial	23
4.2 System Overview	23
4.3 System Architecture	25
4.4 Module Details	26
4.4.1 File System Module	27
4.4.2 Dialogue Module	28
4.4.3 Level Module	29

CHAPTER 5. EVALUATION	35
5.1 Experiment	35
5.1.1 Procedure	36
5.2 Data Remediation	37
5.3 Data Synthesis	37
5.4 Demographic Analysis	41
5.5 Further Quantitative Analysis	49
5.5.1 Overall Correlations	49
5.5.2 Overall T-Tests	52
5.5.3 Split Correlations	52
5.5.4 Split T-Tests	55
5.5.5 Comparative T-Tests	56
5.6 Sequence Analysis	56
5.7 Usability Analysis	59
CHAPTER 6. DISCUSSION & CONCLUSION	65
6.1 Demographic Differences	65
6.2 Correlations	65
6.3 Addressing Negative Correlations	67
6.4 Knowledge Gain T-Tests	68
6.5 Sequence Discussion	68
6.6 Usability Discussion	69
6.7 Summary of Findings	70
6.8 Answering Research Questions	71
6.9 Addressing Shortcomings	72
6.10 Direction of Future Work	74
REFERENCES	75
APPENDIX A. EXPERIMENT DOCUMENTS	78
A.0.1 Parent Consent Form	79
A.0.2 Student Consent Form	81
A.0.3 Pretest	83
A.0.4 Posttest	87
A.0.5 Demographic Survey	91
A.0.6 Usability Survey	93

LIST OF TABLES

Table	Page
3.1 Target Concepts	16
5.1 Testing Activities	37
5.2 Demographic Counts	38
5.3 Quiz Averages	41
5.4 Gender T-Test	44
5.5 Gender ANOVA Test	45
5.6 Computer Access T-Test	45
5.7 Educational Game T-Test	46
5.8 Educational Games ANOVA Test	46
5.9 Computer Hours T-Test	47
5.10 Computer Hours ANOVA Test	47
5.11 Grade T-Test	48
5.12 Region T-Test	49
5.13 Overall Correlations	50
5.14 Overall T-Test	52
5.15 High-Scorers Correlations	53
5.16 Low-Scorers Correlations	53
5.17 High-Scorer T-Test	56
5.18 Low-Scorer T-Test	56
5.19 Comparative T-Test	56

LIST OF FIGURES

Figure	Page
Figure 3.1 Game Flow 1	15
Figure 3.2 Game Flow 2	17
Figure 3.3 Example File Structure	19
Figure 3.4 Game Flow 3	22
Figure 4.1 Minimum Difficulty Level	24
Figure 4.2 captiontext	25
Figure 4.3 Inter-Module Communication	26
Figure 4.4 File System Module UML	28
Figure 4.5 Dialogue Module UML	29
Figure 4.6 Level Module UML	30
Figure 5.1 Knowledge Gain Distribution Comparison	40
Figure 5.2 Overall Performance Histogram	42
Figure 5.3 Maximum Difficulty Histogram	43
Figure 5.4 Maximum Difficulty Scatterplot	43
Figure 5.5 Gender Performance Hist	44
Figure 5.6 Computer Access Performance Histogram	45
Figure 5.7 Educational Games Performance Histogram	46
Figure 5.8 Computer Hours Performance Histogram	47
Figure 5.9 Grade Performance Histogram	48
Figure 5.10 Region Performance Histogram	49
Figure 5.11 Overall Performance Scatterplot 1	51
Figure 5.12 Overall Performance Scatterplot 2	51
Figure 5.13 Overall Performance Scatterplot 3	51
Figure 5.14 High Scorer Performance Scatterplot 1	53
Figure 5.15 High Scorer Performance Scatterplot 2	54
Figure 5.16 High Scorer Performance Scatterplot 3	54

Figure 5.17	Low Scorer Performance Scatterplot 1	54
Figure 5.18	Low Scorer Performance Scatterplot 2	55
Figure 5.19	Low Scorer Performance Scatterplot 3	55
Figure 5.20	Mistake Histogram	57
Figure 5.21	Difficulty vs. Level Plot	58
Figure 5.22	Time Spent Plot	59
Figure 5.23	Overall Usability Histogram	60
Figure 5.24	Usability Categories Histogram	60
Figure 5.25	Engagement Keywords	61
Figure 5.26	Trouble Keywords	62
Figure 5.27	Improvement Keywords	62
Figure 5.28	Engagement Categories	63
Figure 5.29	Trouble Categories	63
Figure 5.30	Improvement Categories	64

This thesis is accepted on behalf of the faculty of the Institute by the following committee:

Ramyaa

Academic and Research Advisor

Rita Kuo

Subhasish Mazumdar

I release this document to the New Mexico Institute of Mining and Technology.

Wesley Camphouse

March 18, 2025

CHAPTER 1

INTRODUCTION

Today's society is very computer-oriented, with electronics being used for a wide variety of applications. Every day, experts make use of computers to aid them in their fields. Researchers make use of technology to obtain precise measurements, model natural events, and make data more comprehensible. To make use of these specialized technologies, researchers are trained in their use. While certainly productive, this behavior does raise the question of whether or not the same treatment is given to laypeople regarding more everyday technology?

While everyday people may not use the advanced hardware and software available to researchers, computers are still a large part of their everyday lives. A layperson may use computers to organize their schedule, communicate with others, and even obtain basic information regarding world events. This is true not only of adults, but of children who may still be learning how the world works as well. Given this, it seems sensible to provide some level of training in school regarding the use of computers to ensure that those growing up alongside them know how to use them effectively. Unfortunately, with the wide variety of subjects already battling for student's attention, it may seem infeasible to provide comprehensive training to them on the use of computer technology. Given this, perhaps it would be best to try to provide computer training through computers themselves, rather than relying on analog methods already in use for other topics.

Based on this, an educational game was developed with adaptive difficulty, designed to teach a user about the appropriate use and organization of the digital file system. The game itself is a simple 2D platformer, with the player progressing by navigating from the left side of the screen to the right. A game format is chosen due to the positive effect that these can have on student motivation and engagement [1][2]. Additionally, a game provides a convenient environment for the effective utilization of adaptive learning, a strategy which can have further positive effects on intrinsic motivation and engagement [3][4].

As for why the digital file system was chosen as a target topic, there are several factors that make this advantageous. First and foremost, the file system itself is a fairly simple concept that should work well with a proof-of-concept system such as this. The file system also has well defined characteristics (such as the path between two locations) that may assist in the implementation of adaptive learning. It is also a fairly important concept for laypeople to understand, as

knowing how to properly leverage the digital file system allows users to make more effective use of their computers for both work and personal life. The final reason, however, has to do with the shared digital nature of the file system and the computer game. Simply put, it is possible for the game to interface with the real file system on the machine, allowing the game to require that players use the real file system in order to bypass certain in-game obstacles. This allows the game to provide a unique way for the player to gain experience using the actual file system of their machine.

CHAPTER 2

LITERATURE REVIEW

When conducting research in this vein, it is always necessary to consult previously-existing literature. When doing this, there are a number of aspects involved in this project that should be researched. Since this is a paper about a technical system, it makes sense to focus on other technical papers. However, this project is also meant to improve technical education, so some level of research from educational sources is also useful. Due to this, this paper will reference a mix of technical papers and educational ones, in order to fully cover the topics of this project. Despite this mix, this paper is still technical due to the nature of the system.

2.1 Computer Literacy

While digital literacy and computer literacy are similar topics at a surface level, there are some key differences that set them apart. Digital literacy is largely defined as the skills needed to find, evaluate, and share information gathered from digital sources [5][6]. This includes the skills needed to participate in social networks, find digital information, and create digital information. In contrast, computer literacy is composed of the skills needed to actually interface with a computer [7]. This largely involves the development of ICT skills, such as the creation and management of files, use of email systems, navigating the internet, and online communication [8]. While this distinction is important, the similarity and overlap between the two concepts makes it likely that research into one will also have implications for the other. In addition, it has been noted that the concept of digital literacy must be fluid and adaptable in order to account for advancements in the digital world [6]. This makes it possible to extend aspects of digital literacy into computer literacy, and vice versa. Due to these factors, this literature review will consider research into digital literacy in addition to computer literacy. With this understanding, the next point of discussion becomes the importance of digital and computer literacy.

An important aspect of the discussion of computer literacy has to do with the importance of the topic. Looking at existing research, it appears that papers concerning digital literacy are on the rise, suggesting that the topic holds some importance [5]. Moving into more specific papers, a 2021 study from Switzerland

investigated the ICT skills of sports students during the COVID-19 pandemic [8]. This study surveyed these students, gathering a total of 360 responses regarding each student's use and attitude towards various ICT skills. While students reported not using as many ICT skills on campus, they did report fairly extensive use of ICT skills in other contexts. Additionally, those who were found to have more skills reported a better experience with online learning, something that was essential in the height of COVID-19. Similar findings regarding online learning have also been found in other studies, noting that greater ICT skills lead to better learning outcomes [9][10]. Based on these findings, it is clear that computer literacy skills are important enough to warrant investigation into any possible deficiencies.

Unfortunately, studies investigating computer and digital literacy seem to indicate that public skills are not meeting the requirements of the modern world. From certain findings, it seems that a portion of students are not confident with the use of computers [9]. Combined with student reports that they learn vital computer skills in the classroom, it is possible that not enough focus is being given towards teaching these skills [8]. Even with those students who are confident, it has been shown that their actual skill level tends to be somewhat behind what they believe their understanding to be [11]. While these findings are focused on the computer skills of students, there have been studies that indicate people in the professional world are behind as well.

Based on recent research, it seems that those in the nursing field lack some of the digital and computer literacy skills required to effectively perform in their jobs [12]. In a case where a new electronic system was being installed at a hospital, it was found that 1/3 nurses working there were anxious about the introduction of the new system. Additionally, just under half of the nurses polled reported that they were completely confident with the use of computers, with the remainder reporting themselves as moderately confident or lower. This is somewhat alarming for a field such as nursing, where computers are regularly used to track data on different patients. As a matter of fact, nurses who were polled in a separate event reported computer literacy skills to be vital to their roles, including ICT skills such as file system management [13]. With just over a quarter of these nurses having gone through some form of formal computer training, it seems likely that more needs to be done for computer literacy education [12][13]. Before investigating how more can be done for computer literacy education, however, it may be important to further consider some of the consequences of high and low levels of computer literacy.

A number of studies have been conducted investigating the effects of computer literacy. As discussed previously, it has been found that students with a higher level of computer and digital literacy tended to experience better learning outcomes in online classes [8][10]. Looking further than this, it seems that these effects may not be limited to students. It has been noted that 21st century skills are vital to workers in the modern age, with computer literacy skills being a part of that [14]. This finding indicates that the level of computer literacy in the general populace could have far-reaching implications as the world relies more and more on digital technology. This is true not just in the workplace, but

also in the personal lives of everyday people. Based on certain findings, it seems possible that digital literacy plays a part in how likely a person is to believe misinformation [15]. This means that improving the digital literacy of the public may help people be more informed of happenings in the world. From all of these points, it is apparent that computer and digital literacy have far-reaching consequences for the world. Combined with the possible deficiency in these skills, it becomes necessary to find some means of improving the digital literacy of the public. From here, it is necessary to determine what means to use to improve digital literacy education. Fortunately, it is possible that the creative solution of game-based learning (GBL) could be a viable method towards this end. It has been shown that playing computer games can reduce anxiety surrounding computers and improve computer literacy [7]. Since these seem to be inherent effects of computer games, it is worth investigating how these games could be used to better facilitate the growth of computer and digital literacy.

2.2 Gamification and GBL

Gamification and GBL are topics that have been studied for some time, but are still being iterated on and improved. While the two concepts do have many similarities, there are key differences that distinguish them from one another. Gamification is the act of adding game elements to an existing, serious structure [1][2]. These game elements can include leaderboards, points, levels, and rewards. Game-based learning, however, is learning that is achieved through playing a serious game[1]. A serious game is a game made to be used in real-world contexts, such as education. Both gamification and GBL can be applied to education, with the goals of increasing engagement and motivation. By increasing engagement and motivation, it is hoped that students will experience a positive impact on learning. With an understanding of what gamification and GBL are, the next step is to investigate how effective they can be.

Since this is a framework that is used for teaching, it is vital to evaluate how effective it is. From existing research, there are a few different positive outcomes that it does seem to have [1][2].

- A positive effect on motivation.
- A positive effect on engagement.
- Providing clear, achievable goals.
- A positive effect on behavioral outcomes (having students follow instructions).
- A positive effect on cognitive outcomes (learning).

Notably, even teachers who utilized gamification and GBL noted some positive effects, such as extra flexibility and improved relations with students [2]. While these positives are appealing, there were also a number of downsides that were noted. The primary noted downsides were:

- Lack of universal appeal.
- Skepticism.
- Infrastructure requirements.

Despite these limitations, overall sentiment towards gamification and GBL seems to be positive. As a matter of fact, sentiment seems to be one of the primary advantages of these methods, with some noting how this can allow education to better respond to student cognitive and emotional states [16]. This is largely due to the ability of these educational games to foster a more fun learning environment, allowing students to learn in a more self-directed way. Through this, students feel better about their learning which has the potential to improve their knowledge gain. For these reasons and others, some have noted a need for more research into effective gamification and GBL strategies, even using them alongside other concepts such as adaptive learning [17].

One more specific benefit of GBL is how it can allow students to learn through practice, rather than simply through lecture. An example of this can be found in a 2010 study by Getchell et al., where a group of archaeology students were given a game called LAVA in which they learned about excavation scenarios [18]. In the game, players were allowed to experience the process of excavating the Acropolis of Sparta. In particular, the game focused on leveraging three specific concepts to help teach students.

- Progression, to keep students engaged by having them complete smaller tasks.
- Random behavior, to maintain challenge and have unpredictable elements.
- Selective revelation, to control how players learned of new game mechanics.

By leveraging these different design aspects, the game was able to effectively provide students with a learn-by-doing experience. It is worth wondering, however, why this was such a large goal of the designers. Why is learn-by-doing a strategy that is strived for?

The learn-by-doing approach to learning was largely spearheaded by John Dewey [19]. When outlining this concept, Dewey based it on four philosophical stances:

- Examining things based on practical consequences (Problem solving, learners do what will bring them to the correct outcome).

- Knowledge acquisition is inherently interactive (It is impossible to properly learn about something without interacting with it).
- The importance of context (To properly consider practical consequences, learning must be grounded in a context).
- Fallibilism (nothing can be known for certain, so mistakes are fundamental to learning).

These stances are often respected in technical classes, where the need for students to gain practical knowledge is heavily emphasized[19]. As an example of this, a 2023 study followed an electrical engineering class that was made to heavily use the learn-by-doing approach to learning[20]. In this class, students were given protoboards that they were able to plug real electrical components into, alongside simulated boards and components. Overall, students reported satisfaction with this learning style, stating that it granted them a better understanding of how theory relates to practice. As seen with the LAVA system, digital games can serve as an effective means to implement this learn-by-doing teaching method [18]. From here, the question becomes how GBL can be effectively implemented.

To determine how GBL can be implemented into a system, it is helpful to consider how other projects have achieved this goal. Overall, there seems to be a number of different design concepts that can be used in order to create a system that leverages the benefits of GBL [1].

- Provide clear goals to the user to maintain motivation.
- Allow the user to set up their own personalized goals.
- Provide immediate feedback on player actions.
- Provide positive reinforcement for desired actions.
- Allow users to make social comparisons among peers.
- Connect users so that they can work towards a common goal.
- Adapt to user input and actions.
- Nudge users toward the correct course of action.
- Provide users with varying means to achieve their goal.
- Use the game context to simplify content.

As can be seen, there are a wide variety of ways that educational games can provide players with a more effective learning experience. In particular, these measures have been noted to help students have more positive emotions regarding learning, feel in control, take a more active role in their learning, and maintain the flow of proper challenge [16]. All of this shows how GBL itself can have a beneficial effect on the learning experience. However, there may be other systems and methods that can work in tandem with GBL to further improve learning.

2.3 Adaptive Learning

Adaptive learning (AL) is a concept that has many potential benefits for a GBL based system. AL is, most simply put, what happens when the method of teaching is adapted to the student [21]. Research into this has been on the rise in recent years as a potential way to improve learning outcomes for students [22][23]. When implementing a system that combines GBL and AL, there are a few different levels of adaptability that can be chosen from [16]:

- Pre-game, where the experience is adjusted before the student actually plays the game.
- Competence-based in-game adaptivity, where the system adapts during play based on learning progress.
- Psychological in-game adaptivity, where the system adapts during play based on the psychological state of the student.

In addition to these adaptation levels, there are also two axes of adaptability [21]:

- Style, where the game adapts to the preferred learning style of the student.
- Behavior, where the game adapts based on certain metrics collected from the student, such as scores.

In order to properly evaluate the potential of an AL system, it is necessary to look into how effective this type of system tends to be. Based on previous findings, it seems that AL systems are effective to some degree. While AL does not seem to increase student scores in examinations by a meaningful amount, students who used AL systems reported higher levels of satisfaction and motivation, alongside lower levels of stress [3][4]. In addition, classrooms that implemented AL spent less time reviewing content that had already been covered, leaving more time to focus on new concepts [3]. Finally, some cases found it possible to automate AL systems, providing a cheaper means of adapting to students [4]. While there are drawbacks to this kind of system (such as needing to make many versions of content), this concept still shows enough promise to warrant investigation into its implementation [21].

As previously mentioned, there are different ways that AL systems can be made to adapt to students [21]. Due to the fact that this concept has been successfully applied to online learning, it is obvious that a digital application of this teaching style is feasible [3]. When doing this, it is important that the method of implementation used reflects the goals of the system, namely the improvement of student attitudes. Towards that end, three main models have been outlined as components for an automated AL system [23]:

- The learner/student model, which contains characteristics of the student.

- The content model, which contains the content that is to be taught.
- The instructional model, which handles the actual adaptation performed by the system.

These three models fall into two categories; the adaptive source (the target that informs how adaptation takes place) and the adaptive target (the components that change according to the requirements of the adaptive source) [23]. In this context, the learner/student model is the adaptive source, while the content and instructional models are the adaptive targets. The final component of this type of system is the adaptive engine, which is responsible for altering the adaptive targets to match the needs of the adaptive source.

While this technical definition of an AL system shows promise, there are some aspects that should be noted. Some studies have indicated that more research needs to be done into teacher acceptance of these systems, as that is a vital aspect of their actual implementation [24]. Regarding this, some sources have pointed to the technical aspect of these systems to be something that may interfere with teacher acceptance and adoption [25]. While teachers have noted promise in AL systems, they have also mentioned challenges that stand in the way of its actual implementation. Additionally, teachers expressed a reluctance to allow these systems to evaluate their students, as the system's evaluation methods may not fully align with their personal teaching philosophy. Finally, teachers noted that students seemed to spend too much time trying to resolve technical issues in the system, rather than actually taking part in learning. These notes may indicate that, while AL systems have the potential to be very useful, it may be worthwhile to downsize them and create systems with a smaller scope. A challenge that stands in the way of these systems regardless of their size is the need for many versions of the same content [21]. Perhaps there is another method that can be implemented alongside AL to help mitigate this.

2.4 Automatic Item Generation

While it can be very expensive to make many redundant versions of content (as is required by AL), automatic item generation (AIG) is a concept that may be able to lessen this load [26]. AIG is what the name implies, constituting any system capable of automatically generating new items for evaluations and tests [27]. The items generated by these systems can be any number of things, but are most often individual questions. AIG systems are also already being used in adaptive contexts, such as their implementation within computerized adaptive testing (CAT) [28]. In these tests, a machine selects automatically-generated questions in order to extract the most information about tester performance in as few questions as possible. This is not purely conceptual, CAT and AIG are already being used together to administer multiple-choice tests [26], demonstrating the relevance of this concept. With these systems already being used in the real world, it is worth investigating just how effective they can actually be.

When it comes to the effectiveness of AIG, research indicates that it can successfully fabricate many items of fairly high quality [27]. Due to the automatic and speedy nature of this, AIG systems are able to lower the otherwise high cost of creating many redundant items [26][29]. More specifically, in spite of drawbacks such as low item diversity, AIG was found to be cost effective when used to synthesize 200 items or more [29]. Alongside this, AIG was also found to have a variety of other benefits:

- The quick fabrication of items can allow for a better test turnaround time [29].
- Potentially better item validity due to the required concrete logic [28][29].
- While the exposure of the algorithm can be an issue, the speed at which new questions can be created reduces the impact of leaked questions [27][28][29].
- Combines effectively with CAT to almost totally automate the testing process [28].

Due to these factors, it seems that AIG has the potential to be very effective, especially when combined with adaptive systems such as CAT. With this effectiveness established, the actual implementation of these AIG systems can be investigated.

AIG is a very technical process, with many different frameworks and theories that can be implemented. Despite this, virtually every method has the common core of manually establishing some sort of template item, and then varying that template to produce new items [27][28][30][31]. One vital part of the design process is to select a theoretical foundation and degree of automation [30]. Firstly, there are two types of theoretical foundation that can be implemented:

- The bottom-up approach. This method is less theoretically driven, with test aspects made with little regard for a theoretical model.
- The top-down approach. This method is more theory-based, with questions being designed according to the thought process required to solve it.

Once one of these foundations are established, one of two degrees of automation must be chosen as well [30].

- Item generation guidelines are written out and then manually executed by a human.
- A program is written to follow the item generation rules and fabricate new items automatically.

Next in the design process, the designer needs to decide what type of item model and content identification they will use [27]. While these aspects can be swapped, the pairs presented here represent how they are most often used together.

- 1-layer, weak theory. This is where new items are generated by manipulating a few aspects of the template, largely to provide variety.
- N-layer, strong theory. This is where new items are generated by manipulating many aspects of the template in a conscious effort to alter the difficulty of the item.

From here, it needs to be decided how new items will be developed [31]. One option is the item model approach, where an existing item is altered to create isomorphic variants. The next option is the cognitive design system approach [26]. This approach starts by identifying key features in an item that can be manipulated, based on the cognitive processes that must be utilized to solve the item. From there, these features are manipulated to create new items that require varyingly intense cognitive processes to solve.

If CAT is not also being used, then this is the final step in designing the AIG system. However, if also performing CAT, then the final necessary step is to determine how items should be selected. One of the most prominent methods is item response theory (IRT) [28]. In IRT, an initial level of ability is evaluated or assumed, with the following three steps being carried out in a loop afterwards:

1. Select and present an item that should be ideal within practical constraints.
2. Collect an answer from the participant and calculate a new estimate of their ability.
3. Check to see if the stop criteria have been fulfilled.

With all of those steps completed, an AIG system can be fully developed, even making use of CAT to automate nearly the entire testing process. From this newfound understanding of GBL, AL, and AIG, it is possible to investigate different systems that have employed these concepts and evaluate what aspects of them were and were not effective.

2.5 Similar Systems

In any project, it is important to review the work that has been done by others in the past. Thus far this paper has broadly investigated such projects in order to understand certain systems, but a more detailed review of more specific projects can also be helpful. Ideally, by investigating these existing projects, it will be possible to incorporate successful aspects of their design into a new work. Due to the targets of this project, it makes the most sense to focus on existing works that make use of GBL, are adaptive in some way, or seek to teach aspects of computer literacy.

As a first work to investigate, consider a 2014 study by Nikolaos Pellas in which students in a programming course were given a 3D game in an attempt to

help them learn [32]. The first motivation of this project was to provide students with a GBL experience in order to promote engagement. As an additional means of maintaining engagement, the game was made to be 3D so that students could interact with its components in a way more similar to how they would in real life. Finally, the game was made to be multi-user, meaning that students were able to play it alongside their classmates, leading to further engagement. Once the game had been fully developed with these designs in mind, students played it throughout the course and reported their thoughts at the end. Overall, the game was fairly successful, with students behaving more positively towards their studies. This system demonstrates the use of GBL for digital concepts, as well as showing the importance of immersion.

While the previous example is excellent for investigating GBL being used to teach a digital concept, it was not adaptive in any way. Fortunately, other studies have done more for the use of adaptive systems in educational contexts. A 2013 study was conducted where an educational game was developed in an effort to promote engagement in students [33]. To appropriately maintain the challenge, the designers of this game implemented an adaptive difficulty system. To see what sort of system would work best, the designers implemented three different educational systems where the user was tasked with translating English to Spanish using cognates:

- A Spanish cognates bubble game with adaptive difficulty.
- A Spanish cognates bubble game with non-adaptive, incremental difficulty.
- A written activity where the user converts English words to Spanish using cognates.

After each student had completed their assigned activity, they were surveyed to see how they felt about the activity they had participated in [33]. While students in all activities reported similar levels of engagement, students who took part in the adaptive activity tended to have higher levels of perceived confidence and better learning outcomes. A similar study to this was in 2020, hoping to gather student attitudes towards an adaptive activity [34]. In this study, it was found that students who participated in an adaptive activity tended to have better knowledge retention than those who participated in a more standard, traditional activity. Together, these two studies demonstrate that adaptive systems can be used to maintain an appropriate level of difficulty over the course of an educational game.

The final system to consider is one from a 2019 study that is called BashDungeon [35]. In this system, researchers implemented a unique intersection of computer science education and GBL in an attempt to teach university students about the use of the UNIX operating system. BashDungeon began with a brief tutorial in which students were taught very basic UNIX commands, such as those needed to list the contents of a folder and to move between folders. From there, students were put into a digital dungeon with a similar layout to the UNIX topology, with

the rooms of the dungeon being folders and the items in those rooms being files. Once in this dungeon, students needed to use UNIX commands to navigate it, with different rooms teaching students how to use different commands. Overall, the game taught students how to use many different commands, with more complex commands being introduced as time went on. After the game had been developed, it was tested on first year university students who had not yet taken the UNIX class. Through this test, BashDungeon was found to have successfully taught students how to use various UNIX commands. The success of this project is quite interesting, as it demonstrates an ability for GBL to provide a learn-by-doing experience when teaching digital concepts.

2.6 Research Questions

From this dive into existing projects, a number of different concepts come to light as topics to investigate. First and foremost, it seems that developing another means of promoting computer literacy is justified, based on the importance of the topic and the apparent deficiency. Additionally, it is apparent that GBL can be used to teach a wide variety of concepts, making it a prime candidate to promote computer literacy. On top of this, it seems that adaptive learning and AIG can be used alongside GBL to provide a more dynamic learning experience. All of these concepts have been implemented in previous research to some degree, providing confidence that they can be made to work.

Based on all of these findings, it seems worthwhile to create an educational game that teaches students computer literacy skills, while also leveraging aspects of AL and AIG. Since this would be a fairly unique system, it makes sense to design it as a proof-of-concept more than anything, so selecting a more simple aspect of computer literacy would be beneficial. Towards this end, the digital file system is selected as the concept the game will teach. This is chosen firstly because it is fairly simple, lending itself well to a proof-of-concept design. Additionally, the digital file system has easily defined characteristics, which will help when creating a template item with adjustable traits for the AIG system. Finally, the digital file system is an important concept in everyday computer use, as digital files are used frequently in people's personal and professional lives.

Towards these ends, a number of specific research questions are found to be in need of answers:

1. How can an educational game be made to procedurally generate puzzles about the digital file system?
2. How could this game determine what level should be generated, given the outcome of the previous level?
3. How does the proposed game improve student understanding of the digital file system?

CHAPTER 3

RESEARCH METHODS

With the broad concept of the project introduced, it is possible to properly outline some of the goals and methods that the game should use. This will involve discussing the specific concepts that should be utilized, as well as some idea of how the game may leverage them. Since the goal of the game is to provide players with a greater understanding of how the file system may be used, all of these concepts should be used to further that goal.

Before discussing them in detail, it may be useful to provide a rough outline of the different concepts that will be discussed. The first topic will be the incorporation of game-based learning (GBL), as that is the core of the project. This will involve outlining the broad game concept, and how it will leverage the fundamentals of GBL to teach about the digital file system. The next topic to be discussed will be adaptive difficulty, outlining what it is, how it will be incorporated into the game, and what benefits could be gained. Finally, the use of automatic item generation (AIG) will be discussed. This will involve what an item is in the context of the game, how it will be integrated into the adaptive difficulty, and what benefits it could provide.

3.1 Game-Based Learning

The first thing that should be discussed is also the one that sits most at the core of the project; the incorporation of GBL. Since this project is largely intended to be a proof of concept, the game should be fairly simple to implement. Due to this, the game takes the form of a 2D platformer, a type of game that is relatively simple to make and understand. To further promote simplicity (as well as working well with goals mentioned later on) the game is given short, discrete levels. Each level will involve some manner of file system related puzzle that the player must solve before they are able to move on. Once the player completes the requisite number of levels, the game should congratulate them and end. Since this is a fully structured educational game, rather than applying game elements to an existing learning strategy, this is enough to leverage GBL. Fig. 3.1 illustrates how the game progresses from one point to the next, beginning with an introduction, then leading into a sequence of levels, and finally ending in the exit sequence.

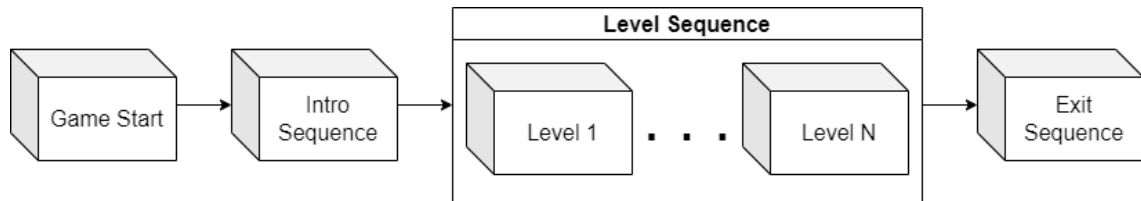


Figure 3.1: Overall flow of the game.

Since the purpose of the game is to teach students about the proper use of the digital file system, this should be reflected as the primary goal of its design. Due to this, the level of mechanical challenge provided by platforming should be rather low, as testing a player's ability to play a platformer is not the goal of the game. In contrast, the file related puzzles should be rather challenging, as this will allow the player to actually gain knowledge from their experience. It is also important to avoid overwhelming the player, however. While a very high level of challenge may be appropriate if the goal of the game were to test the file management skills of a player, the goal of teaching requires that the difficulty be calibrated such that players are challenged but not overwhelmed. Doing this properly should create an environment where players learn to use the file system through practice, bringing them into a flow state where learning is easier. With the goal and broad design of the game established, the next question becomes how to properly incorporate the file system into the puzzles.

Since the game is intended to teach students about the use of the file system via its puzzles, the design of these puzzles is very important. The design of this system should cater to the specific concepts that we seek to teach, especially since this will also constitute the content model that is used for the later integration of adaptive learning. Additionally, it would be ideal if the puzzles were designed in such a way that students can reap the benefits of the learn-by-doing approach to learning. To properly take advantage of this concept, it would be best if students were to directly interact with the real file system of the machine, which is a strange concept for a game. Fortunately, there is an aspect of this system that should make this easier. Since both the file system and the game are digital, it may be possible to leverage this shared nature to give students the full learn-by-doing experience.

Rather than attempting to imitate the file system in the game, the game can require that students interact with the real file system of their machine. To do this, the game would be given access to a specific section within its own files that it would manage. Once this is established, it becomes possible for in-game puzzles to require players to complete certain operations within this managed portion of the file system. The tasks given to the players should relate to the game content in some tangible way, since the puzzles are meant to solve in-game obstacles. To fully illustrate this concept, consider an example level and puzzle that a player might encounter. Within the game, the player encounters a locked door in the middle of a city. The door completely blocks their path, and there is no way to simply bypass it. In order to progress, the player must navigate to the

“city” folder within the game’s managed files, navigate further into the “blockers” folder, and finally delete the “door.txt” file. Once the game detects that the file has been deleted, it then removes the door and allows the player to progress. The core idea is that, by providing a tangible relation between the game environment and the file structure, players will gain a better understanding of file organization. The tasks that they are required to perform (such as navigation within the file system and the deletion of files) should also build proficiency with using the file system. Finally, it is also possible that this process will make players more confident with the file system in real contexts, given their experience gained with its use. A summary of the concepts that the game is intended to teach, as well as what is done to teach them, can be found in table 3.1 This concept does, however, raise the question of how puzzles should be designed, since they need to keep the player consistently challenged without overwhelming them.

Concept to teach	Facilitation
File System Organization	Have file structure relate to the game environment.
File System Navigation	Have puzzles require the player to navigate to a specific location in the file system.
File System Operations	Have puzzles require players to perform some file system operation.

Table 3.1: Targeted concepts and how they will be taught.

3.2 Adaptive Difficulty

Adaptive difficulty is exactly what the name implies, it is when the difficulty of a challenge adapts to the apparent ability of the user. It is an aspect of adaptive learning, with the intention being that it can keep the student challenged without entirely overwhelming them. If done correctly, this can boost player motivation. The reason that this project implements adaptive difficulty rather than more advanced adaptive learning concepts is for simplicity. Since this is a proof-of-concept project, the smaller scope of adaptive difficulty lends itself well to this application. Recall that adaptive learning can be done based both on student behaviors and the learning style of a student. Attempting to interpret and adapt to the learning style of a student is a complex topic, so the difficulty adaptation featured in this project will be concerned solely with student behavior. This changes the flow of the game from the one found in Fig. 3.1 to instead resemble Fig. 3.2, with the level sequence replaced by a number of possible levels, one of which is selected for the player at any given time. With a broad understanding of adaptive difficulty obtained, it is possible to discuss the implications that this topic has for the game.

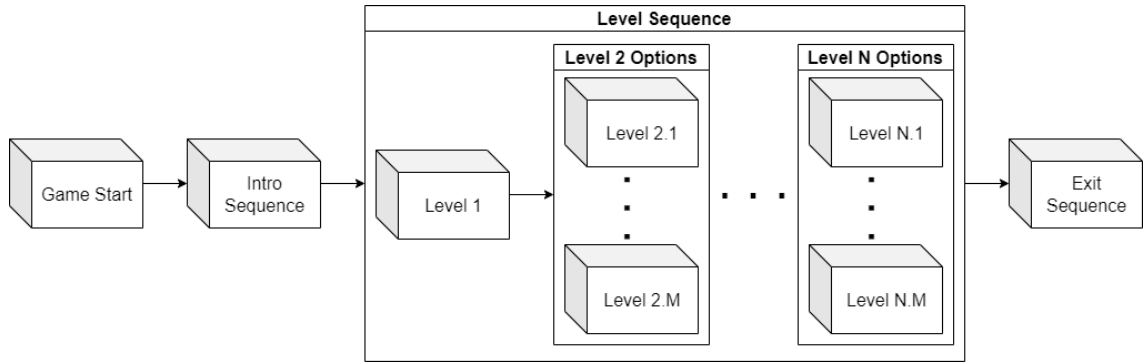


Figure 3.2: Game flow after the introduction of adaptive difficulty.

Before attempting any actual implementation, it is important to outline how exactly it will be incorporated. The first step is to identify what aspects of the game adaptive difficulty will actually affect. This should specifically target the content model that was established in the previous section, so it should serve to better teach file system concepts like organization, navigation, and file operations. Since the main obstacle in the game is the file puzzles, the adaptive difficulty should influence these puzzles in order to keep players engaged. Given the kinds of puzzles that we intend to build, there are a number of different factors that can be changed in order to influence the overall difficulty:

- The distance between the location the player starts at and the actual location of the file (referred to as the depth, maximum of 3).
- How many parts of the file path are not explicitly given to the player (referred to as the unknown path, cannot exceed the depth).
- What type of action the puzzle expects the player to perform (referred to as the type, can be delete, create, or move).

These elements make up the instructional model of the adaptive system. With an outline of what traits the adaptive difficulty system can influence, it is possible to construct a means to calculate the difficulty of a given puzzle. For the type of puzzle, a static amount of difficulty is applied based on the type. For deletion this value is 1, creation adds 4, and movement adds 8. This is based on the ease of deleting a file, the somewhat higher complexity of creating a new file, and then the much higher complexity of moving a file from one location to another. For the depth and unknown path, both values are multiplied by a coefficient and then added to the difficulty of the puzzle. For the depth the coefficient is 0.5, while the unknown path has a coefficient of 1.5. This is based on the idea that adding additional depth to a puzzle increases the difficulty only slightly, while obscuring parts of the path increases the difficulty greatly. The formula for this calculation can be found in (3.1) and (3.2), where T is the type of the puzzle, $Depth$ is the depth, and $Unknown$ is the length of the unknown path. This system

provides a framework for the creation of many puzzles of varying difficulties, with the adaptive difficulty system determining what level of difficulty is most appropriate.

$$\text{Difficulty} = \text{type_wt} + (0.5 * \text{Depth}) + (1.5 * \text{Unknown}) \quad (3.1)$$

$$\text{type_wt} = \begin{cases} 1, & \text{if } T = \text{DELETE} \\ 4, & \text{if } T = \text{CREATE} \\ 8, & \text{otherwise} \end{cases} \quad (3.2)$$

To understand how exactly these formulas work in practice, consider a couple examples using the example file structure found in Fig. 3.3. For the first example, consider the player to be starting in the “forest” folder and needing to delete “boulder.txt” from the “blockers” folder. Additionally, the player is not told the path to that folder, instead needing to locate it on their own. Since “blockers” is only one folder down from “forest”, the depth of this puzzle is one. Since none of the path is given to the player, the unknown path length is also one. Finally, since this puzzle requires a deletion operation, the type_wt is one. From these factors, the difficulty of the puzzle can be calculated as $1 + (0.5 * 1) + (1.5 * 1) = 1 + 0.5 + 1.5 = 3$. As a second example, suppose that the player needs to create a file named “bridge.txt” in the “spanners” folder, and that the player starts in the “game_root” folder. The player is told that they will need to navigate through “level/forest” to achieve this, but is told that they will need to go through one additional, unknown folder. Due to the distance between the starting and ending folders, the depth of this puzzle is three. However, since two components of the path are given (“level” and “forest”), the unknown path length is only one. Finally, since the puzzle requires a file to be created, the type_wt is four. This allows the difficulty to be calculated as $4 + (0.5 * 3) + (1.5 * 1) = 4 + 1.5 + 1.5 = 7$. These two examples should sufficiently demonstrate how the difficulty of a puzzle can be calculated from the factors within.

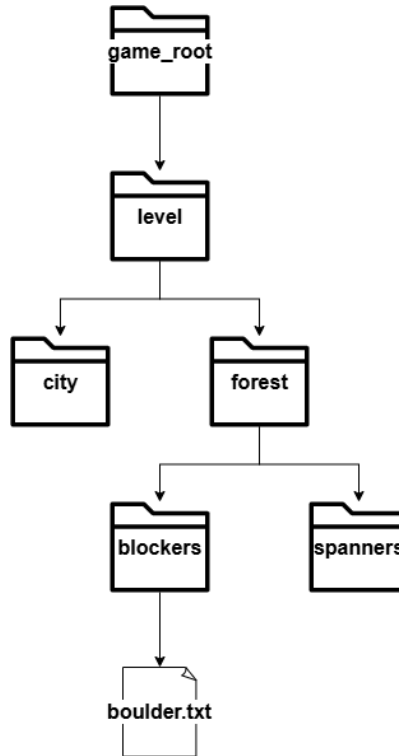


Figure 3.3: File structure used in the difficulty calculation example.

An additional aspect to consider is the measurement of student performance. To construct the student model, it is necessary to both construct a means of measuring student performance and also determine how that influences the desired difficulty of the system. Since the system is expecting players to perform specific operations on the file system (such as deleting a specific file), each puzzle can define any other action taken as a mistake. These mistakes could include deleting the wrong file, making a new file for no reason, or even deleting an entire folder. With this definition, student performance can be measured based on the number of mistakes that they make before performing the correct action. It may also be possible to measure student performance based on time spent in a puzzle, but without access to a “normal” completion time this metric is unreliable. For this reason, student performance will be measured based on the number of mistakes made. This conceptualization of mistakes also allows for a new end state to puzzles, that being the fail state. Obviously, a puzzle will end if the player successfully performs the expected action. However, it is also possible to end a puzzle in a fail state if the player makes more mistakes than is permitted. Since it is a number that is often used in this kind of context, students will fail a puzzle if they make three or more mistakes.

This outline provides a clearer idea of how player actions influence the desired difficulty of the system and, by extension, the puzzle selection. If the player performs the desired action, advance them through the possible difficulty levels based on the number of mistakes that they made before arriving at the correct

solution. If instead a player makes three mistakes, end the puzzle prematurely and lower the difficulty of the next puzzle. Based on this outline of the desired behavior, we can obtain the formula found in (3.3), where N is the new difficulty and D is the difficulty of the previous puzzle.

$$\text{Target Difficulty} = \begin{cases} D + 0.2D * (3 - \text{Mistakes}), & \text{if Mistakes} < 3 \\ D - 0.2D, & \text{otherwise} \end{cases} \quad (3.3)$$

To demonstrate the use of this formula, it may be useful to consider a couple of examples. Suppose that the player has successfully completed a puzzle with a difficulty of three, and only made one mistake in the process. Since the player made fewer than three mistakes, the first equation of the formula will be the one that is used. Based on this, the target difficulty of the next puzzle should be $3 + 0.2(3) * (3 - 1) = 3 + 0.6 * 2 = 4.2$. As another example, suppose that a player failed a difficulty ten puzzle by making three mistakes. Due to the number of mistakes, the second equation will be used. Because of this, the target difficulty for the next puzzle will be lowered to $10 - 0.2(10) = 10 - 2 = 8$. These two examples should demonstrate how the adaptive difficulty system can adjust the target difficulty according to player ability.

This replaces the standard progression from one difficulty to the next with an adaptive system, keeping players engaged throughout the game by tuning the difficulty to their specific abilities. Unfortunately, this adaptive difficulty system requires there to be a rather large number of possible levels since it needs at least one level for each difficulty. The manual creation of these levels would be rather tedious, so it is worthwhile to find some means of automatically creating more levels.

3.3 Automatic Item Generation

Automatic Item Generation (AIG) allows for the game to create new puzzles and levels as they are needed, no longer requiring puzzles to be manually created by a human. This not only eases the implementation of the game, but also has the potential to make the game more engaging as the pseudorandom creation of puzzles and levels can increase variety for students. This can also prevent cheating, since sharing answers with other students means little when the puzzles and levels presented change from one student to the next. Overall, AIG is a very promising concept if it can be properly implemented into the game. The next question to answer is how this can be achieved.

Due to the design of the game outlined so far, it is necessary for the AIG system to work with the adaptive difficulty to maintain motivation and engagement. The first step toward this goal is to define what an item is in the context of the game. The obvious answer in this case is that an item is a level in the game. However, since only the file puzzle is a significant factor in the difficulty of a level, the puzzle is the only factor that the difficulty needs to actually manipulate. Based

on this fact, we separate items into two distinct parts. The first part is the puzzle, which is the file operation that the student needs to complete in order to move through the level. The second is the playspace, which is the digital space in which the player is able to control their character.

For the puzzle, the AIG system should manipulate the factors outlined by the adaptive difficulty system (depth, unknown path, and type) in order to create a puzzle of the desired difficulty. Since this involves the manipulation of multiple different factors, this system is an N-layer item model. Additionally, since this is being done in a conscious effort to modify the difficulty of the puzzle, it is using strong theory content identification. Once the AIG system is informed of the target difficulty by the adaptive difficulty system, it needs to somehow manipulate the different puzzle factors to produce a puzzle of the target difficulty. Since there are only a few factors to manipulate, the simplest way to do this is to simply use a brute force approach, generating every possible puzzle and then selecting the one that is closest to the desired difficulty. With the puzzle generated, the only remaining step for the AIG system is to create the playspace.

Since the playspace does not influence the difficulty, the only reason it is part of the AIG system is to provide variety to the player. This means that the only actual constraint on the playspace is that it has to make sense given the puzzle that has already been generated. To facilitate this, it is necessary to define a number of factors that can be varied from playspace to playspace. The first thing that can be varied is the apparent setting that the level takes place in (such as city, forest, or lab). This can simply be varied randomly, with the specific folder names in the puzzle being changed to match the setting. Additionally, the obstacle that blocks the player from progressing can also vary, although with more constraints than the setting. Firstly, the obstacle has to make sense with the type of action that the puzzle expects. As an example, if the puzzle expects the player to create a new file, then the locked door example from earlier will not work. This is because there is no clear reason why creating a new file would cause a door to disappear, while deleting a file named "door.txt" is a clear reason for the door to be removed. While these two factors may not seem like much, varying the setting and the obstacle allows for a wide variety of different playspaces to be created. Through this, the AIG system and adaptive difficulty system can work alongside the game to create a learning experience that facilitates motivation and engagement in students. This brings the structure of the game to its final form found in Fig. 3.4, with the levels now being created on the fly to meet student needs.

3.4 Summary

After covering the use of GBL, adaptive difficulty, and AIG, it should be clear to see how this game is intended to work. The game itself provides the overall structure, as well as being the main interface through which students interact with the system. The game will consist of a short introductory sequence, a series of levels and puzzles, followed by a brief outro sequence congratulating the

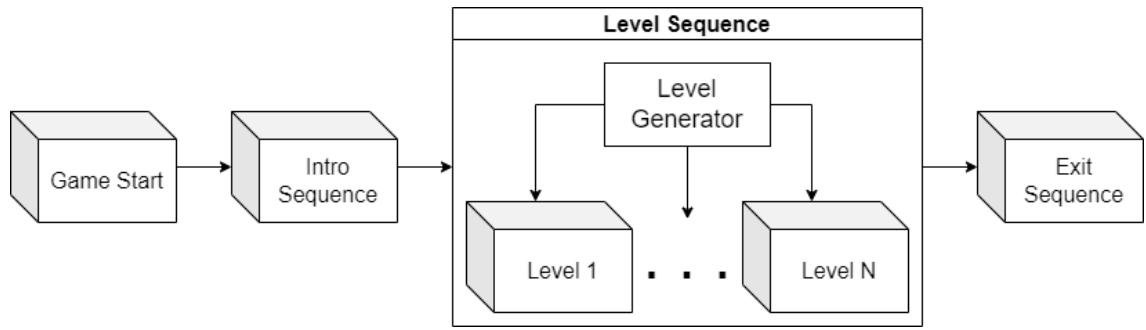


Figure 3.4: Game flow after the introduction of AIG.

player and ending the game. Each level will consist of a short area that the player can control their character in, with an obstacle that requires that level's puzzle to be solved. Each level's puzzle will require the player to perform some specific file system operation, such as creating a new file or deleting an existing one. This puzzle series is where the adaptive difficulty and AIG systems come into play as well. The adaptive difficulty system monitors player performance, constantly updating what the appropriate difficulty is as the player moves through the game. This is done by monitoring how many mistakes the player makes; players who make fewer mistakes will advance faster than those who make more mistakes. Those who make many mistakes may even advance backwards in difficulty for periods of time, giving them the opportunity to recuperate in an easier environment. This strategy requires a large number of puzzles and levels, which the AIG system is charged with creating. In the AIG system, each level is split into two components; the puzzle and the playspace. The puzzle is generated based on the desired difficulty given by the adaptive difficulty system, while the playspace is changed simply to provide variety. All together, these components have the potential to create an experience that keeps players engaged and motivated by maintaining an appropriate challenge, allowing them to learn about the digital file system in a unique way.

CHAPTER 4

SYSTEM

4.1 Tutorial

Before playing the actual game, students went through a brief tutorial program. This program ensured that the students had a basic knowledge of the file system, such as what a folder is. The bulk of the tutorial is a sequence of pages, with each page introducing the player to a new concept. The first few of these pages are used to introduce the player to the file system as a concept, explaining it as a tree structure. From there, the player is introduced to the concept of a file path, with a small challenge being given to ensure that they understand. The remainder of the pages are dedicated to making the player more comfortable with the file system, with one final challenge tasking them with finding a file in the real file system. Through this, the tutorial is able to ensure that players have a basic understanding of the file system before playing the game. With this established, it is possible to discuss the actual game.

4.2 System Overview

Before discussing the implementation details of the system¹, it would be best to more broadly discuss what the game is and how it progresses from start to end. First and foremost, the game is a very simple 2D platformer made with version 2020.3.30f1 of the Unity game engine. This style of game allows the player to fully explore their environment in order to navigate the obstacles within. Since the game was developed in Unity, all of the code for the game has been written in the C# programming language.

After logging in to the game, the player is brought into the main level sequence. During this level sequence, the player will be given a number of individual levels where they must cross from the left side of the screen to the right. In each level there is some manner of impassable obstacle in the player's way, which only becomes passable if a specific file system action is performed. This obstacle can take many forms, from a boulder that requires a corresponding file

¹Code for the system, as well as data collected during research, can be found at <https://github.com/coderCleric/Computer-Literacy-Game>

to be deleted before it will disappear to a pit that requires a certain file to be created before a bridge will appear. Ultimately, these puzzles will always require that a specific file be deleted from the system, created in the system, or moved from one location to another.

Now that the structure of an individual level is established, it is possible to move on to how the level sequence plays out as a whole. First and foremost, the level sequence begins by generating a minimum difficulty level (an example level can be found in figure 4.1 below, with the file system location the player enters at being in figure 4.2). Once the player performs the file system action that the puzzle expects, or performs three actions that it does not, the current level terminates and the player is moved on to the next. The new level that is generated is created based on the difficulty of the previous, as well as how many mistakes the player made on the previous level. This process repeats until the player has gone through ten total levels (successfully or not) or until they have successfully completed the most difficult level. After this, the player is given a brief ending dialogue and closes the game.



Figure 4.1: An example minimum difficulty level.

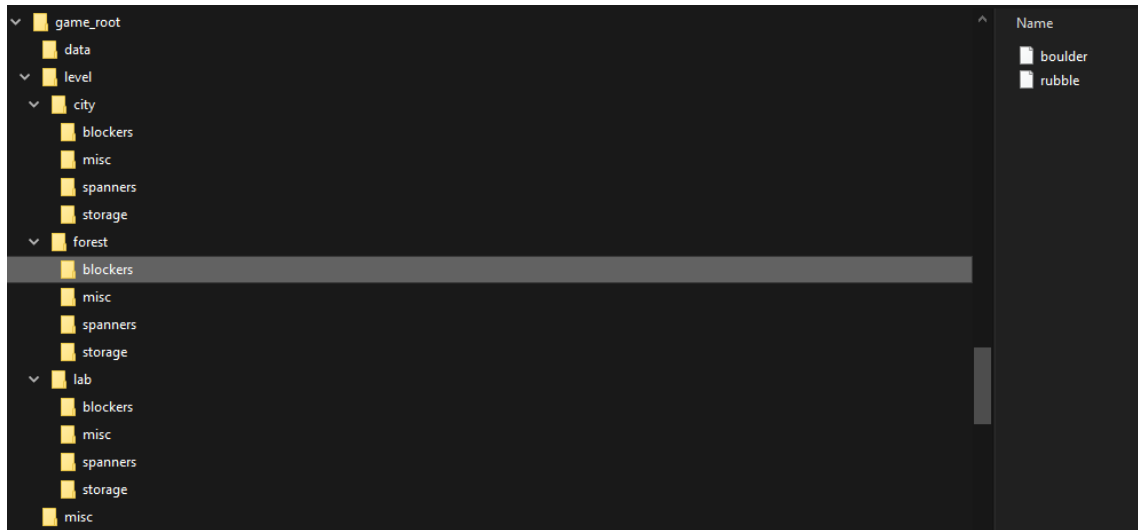


Figure 4.2: Where the player enters the file system in that difficulty one level.

4.3 System Architecture

In any somewhat complex digital system, a very important aspect of the system architecture are the modules that go into the final product. This is no different for the game implemented here, which has a number of different modules:

- The player module, which controls player inputs.
- The level module, which controls level generation and management.
- The dialogue module, which controls how dialogue is presented to the player.
- The file system module, which performs the interaction with the file system of the machine.
- The database module, which controls the flow of information to the testing database.

While there are system aspects that exist outside of these modules, they are handled by the Unity game engine and will not be discussed here.

Since this system is limited to only five modules, the inter-module relations among them are fairly simple. To discuss how different modules interact, focus will be put on the outgoing communications that come from each module. For the player module, the only outward communication that it features is when it informs the level module that a player has successfully completed a level by entering the designated area. The dialogue module tells the player module not to allow player actions if dialogue is playing, and also informs the level module

when certain key dialogues are complete. The file system module communicates with the level module to determine what the file system state is. The database module does not have any outgoing communications, only being given instructions by other modules.

The final component is the level module, which has many more outgoing communications than any other module. The first among these is ensuring that the player module knows where the player needs to start at the beginning of any level. It also sends dialogues to the dialogue module to be shown to the player, as well as querying the file system module for the file system state. Finally, the level module tells the file system module what file system action to expect, and also prompts the database module to send information to the database. A diagram showing how different modules communicate with each other can be found in figure 4.3. With an understanding of how modules interact to make a functioning system, it is possible to discuss the specifics of the more important modules.

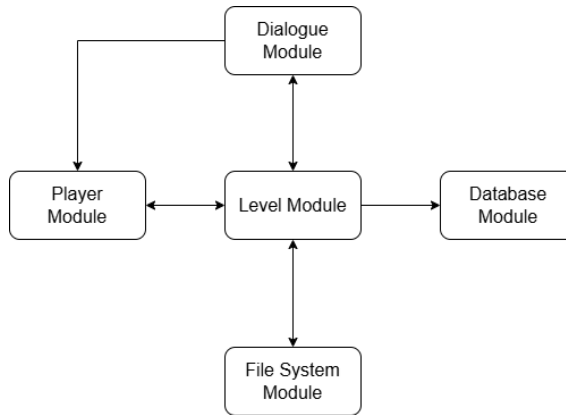


Figure 4.3: Inter-module communication. Messages are sent by the modules at the start of arrows to those at arrow termination.

4.4 Module Details

While there are five total modules that make up this system, it is only worth discussing those that are most important to the file system puzzles and the adaptive system. Towards this end, the player module will not be included in this discussion, as it only concerns the actual movement of the player. The database module will also not be included as it only exists for testing purposes, with the system being able to function without it. This leaves only three modules to discuss in detail: the file system module, the dialogue module, and the level module. While all three are vital, the level module will be discussed in even greater detail due to its importance to the adaptive difficulty and AIG systems. Since it lies at the core of the puzzles and is essential to understanding many of the other systems in the game, the file system module will be discussed first.

4.4.1 File System Module

Since the concept of the game involves interactions with the real file system of the computer, some means of communicating with the file system is required. For ease of use, this communication takes the form of the file system module, an abstraction that other modules can use to more easily interact with the real file system. Much of how this module functions involves leveraging the child/parent relationships present in the Unity game engine. With Unity, game objects can be created to represent different files and folders, with the parent/child associations between them being used to create a mock file structure. This allows the system to avoid creating a new, random file structure for every puzzle, instead using a defined structure represented by game objects and placing the player at a different location within it depending on the needs of that particular puzzle. From there, different classes can manage the state of the actual file system based on the game object structure. To make this process easier, a number of different scripts were created:

- `FSCComponent`, `FileComponent`, and `Foldercomponent` all represent members of the file system, and manage their counterparts in the real file system.
- `FileStructure` represents the abstract file system as a whole, managing the other classes and objects that belong to this module.
- `FileStructureState` is a data structure made to carry information about how the abstract file system relates to the real one.
- `FolderOpener` holds miscellaneous data needed by the module, and also facilitates the player's ability to open the real file system at a specific location.

A UML diagram showing how the classes in this module relate to each other can be found in figure 4.4. Moving on, the next module to discuss is the one responsible for displaying dialogue to the player.

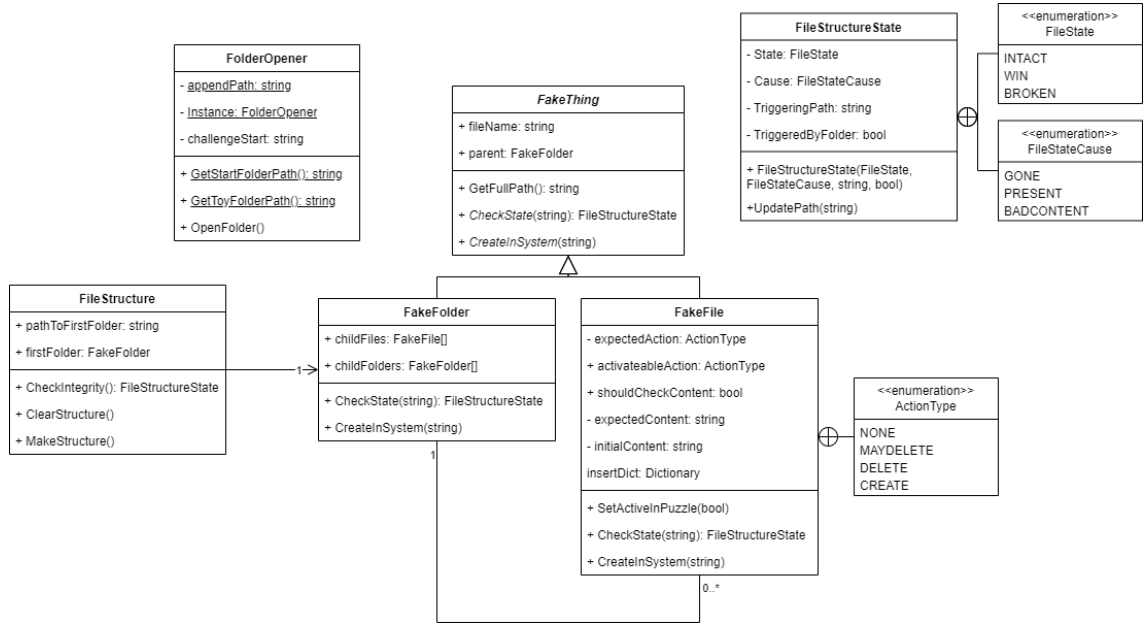


Figure 4.4: UML diagram of the file system module.

4.4.2 Dialogue Module

As previously noted, the dialogue module is responsible for displaying dialogue to the player and ensuring that the game flows smoothly regardless of when it is displayed. While this may seem simple enough, a deeper discussion is warranted as this is the main way that the player receives information regarding the different puzzles that they encounter. Since the puzzle generation utilizes randomness, the dialogue presented to the player has to be generated to fit that as well. Overall, this means that the dialogue module is notably more complicated than it would seem to be on first inspection.

To achieve its goals, a few different classes are included in the dialogue module:

- The DialogueChain class, which stores the dialogue that is later shown.
- The DialogueManager class, the primary class of the module, and the class that actually displays dialogue.
- The PuzzleIntroBuilder class, a static class that makes the introductory dialogue for each puzzle.
- The EndGameDialogueBuilder, a static class which creates the dialogue that is displayed when the player finishes the game.

A UML diagram showing how these different classes communicate with one another can be found in figure 4.5. With this discussion complete, it is possible to discuss the most important module to the game.

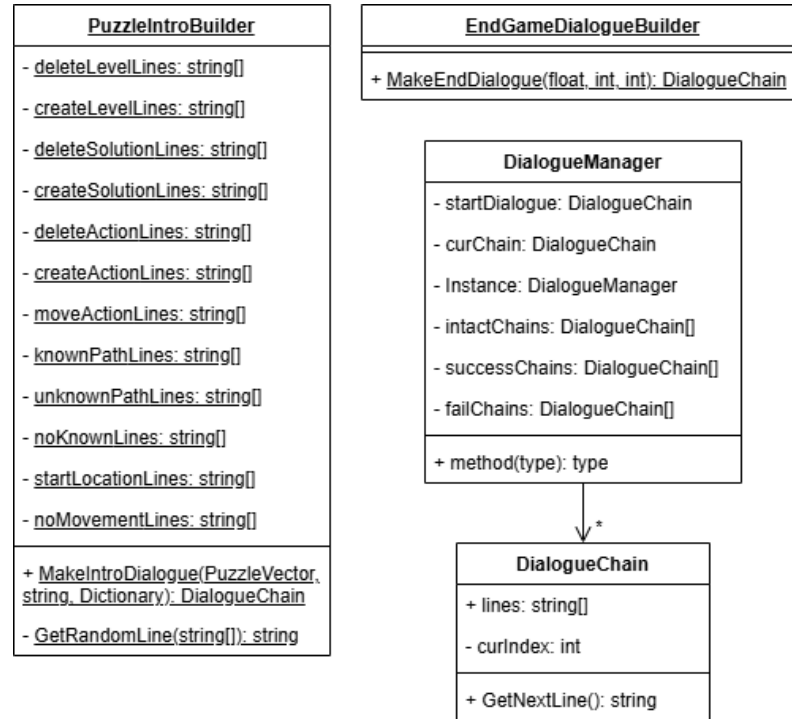


Figure 4.5: UML diagram of the dialogue module.

4.4.3 Level Module

Having discussed the simpler modules, it is now possible to properly understand the level module. The main duty of this module is to generate and manage the various levels and puzzles that the player will encounter throughout the game. Overall, this module is fairly complex, featuring three classes in total:

- **PuzzleVector**, which is the representation of a puzzle's different features.
- **PuzzleLevel**, which is the actual level that the player is able to control their character in.
- **PuzzleManager**, which is the most central class of the entire system, controlling how puzzles are generated and managing the game flow.

A UML diagram demonstrating how these different classes fit together into a cohesive module can be found in figure 4.6. While a broad outline of how

this module is helpful, the PuzzleManager class is important enough to the AIG system to be discussed in detail.

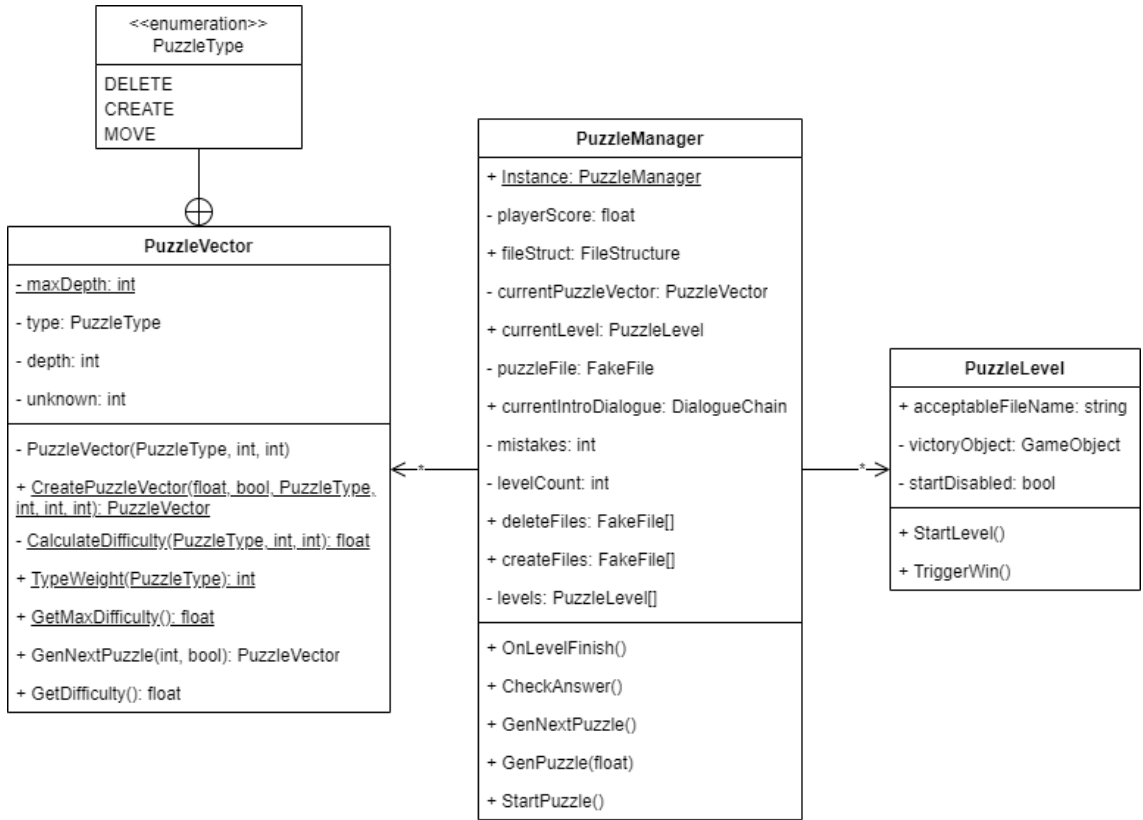


Figure 4.6: UML diagram of the level module.

PuzzleManager The **PuzzleManager** class is responsible for the generation of puzzles and levels, transitions from one level to the next, and overall game flow. It ensures that selected levels make sense with the generated puzzles, as outlined in the research methods. As this might suggest, this is the longest and most complex class in the game, so there are many aspects to go over. Beginning with the important properties, there are several that are worthy of note:

- A reference to the FileStructure instance.
- A reference to the PuzzleVector representing the current puzzle.
- A reference to the current PuzzleLevel.
- A reference to the FileComponent that is the target of the required puzzle action.

- A reference to the DialogueChain being used to introduce the current puzzle.
- An integer counting how many mistakes the player has made in the current level.
- An integer counting how many puzzles the player has encountered thus far, including the active puzzle.
- An array of FileComponents containing all abstract files that can be the target of deletion puzzles.
- An array of FileComponents containing all abstract files that can be the target of creation or movement puzzles.
- An array holding all PuzzleLevels that can be used.

These properties are all integral to the operation of this class, and make it possible for the methods that it contains to function correctly.

Entering the methods of this class, there are a number of methods that are not relevant to the generation of new levels and puzzles and, as such, will not be detailed here. The first method that is relevant to this discussion is the method that handles the completion of a level. This method is specifically called when the player completes a level successfully, not when they are forced to leave a level due to making too many errors. The first step in this method is to check if the player has hit a termination condition for the game, that being if they have completed ten levels or if they have just completed the level of the maximum difficulty. If one of these termination conditions has been met, then the player is put into the outro sequence. Otherwise, the method to generate a new puzzle and start it (both of which will be detailed later in this section) are both called. This ensures that the player is able to continuously move from one level to the next.

The next method to discuss is the one responsible for checking whether or not the player has completed the required file system operation. This method is called whenever the player presses the button to check the state of the file system. This method is somewhat complex, as it handles the logic of player actions and also provides the player with feedback on the action that they took. The first step of this algorithm is to prompt the FileStructure instance to check the state of the file system. If the state is broken, then the number of mistakes made at this level is increased by one. Additionally, if the number of mistakes is less than three after this addition, then the player is informed of their mistake. If the number of mistakes is equal to or greater than three, then the database is notified of the puzzle completion, the player is informed that they failed this level, and they are moved on to a new level. If the state of the file system was intact, then all that happens is that a brief dialogue is played informing the player that nothing happened. Finally, if the state is a win state, then the database is informed of the level completion and the player is congratulated. This also calls the method

to trigger the obstacle in the current level, allowing the player to proceed to the right side of the screen. This all allows this method to correctly evaluate player actions within the file system. Since this method is short but somewhat complex, pseudocode for it can be found in algorithm 1 below.

The next two methods to discuss will be discussed together due to their similarities. These are the methods used to generate a puzzle and to generate the next puzzle that the player should encounter. The primary difference between these two methods is that the one to generate a puzzle takes a desired difficulty as input, while the method to generate the next puzzle uses the current puzzle to inform its generation. The first step of both is to check the maximum number of levels has been reached already, in which case the end sequence of the game will begin. Otherwise, the level count is incremented and a new puzzle vector is generated. For the regular puzzle generation method, this involves calling the static puzzle vector creation method of the PuzzleVector class, passing it the desired difficulty. For the method that generates the next puzzle, this involves calling the generate next puzzle method of the current puzzle vector, providing it with the number of mistakes that the player made, as well as telling it not to apply constraints.

Once the puzzle vector is generated, both methods walk through the abstract file system and set each file to be inactive in the current puzzle. From there, the type of the generated puzzle vector is used to select a random file from either the ones viable for deletion puzzles or the ones viable for creation and movement puzzles. The selected file is then set to be active in the puzzle and, if the puzzle is a move type, the flag for content checking is set to true in the file. Once the file is selected and prepared, the dialogue module is used to create the introductory dialogue for the generated level. Finally, a random level is selected from the stored array that is valid for use with the selected file. This completes the setup of the level, making it ready to start.

Algorithm 1 The algorithm used when a player prompts the system to check their solution.

CheckAnswer()

```
1: if currentPuzzle = null then
2:   return
3: end if
4:
5: state  $\leftarrow$  fileStruct.CHECK_INTEGRITY()
6: if state.State = BROKEN then
7:   mistakes = mistakes + 1
8:   if mistakes < 3 then
9:     DisplayMistake()
10:    fileStruct.MAKE_STRUCTURE()
11:   else
12:     fails  $\leftarrow$  fails + 1
13:     UpdatePlayerScore(-2)
14:     UpdateDB()
15:     DisplayFailure()           ▷ Makes and starts next level once dismissed
16:   end if
17: end if
18:
19: if state.State = INTACT then
20:   DisplayIntact()
21: end if
22:
23: if state.State = WIN then
24:   UpdatePlayerScore( $2^{2-\text{mistakes}} * \sqrt{\text{puzzleDifficulty}}$ )
25:   UpdateDB()
26:   DisplayWin()
27: end if
```

This leaves only one method left to discuss, that being the method responsible for actually starting a puzzle. While this method is not terribly simple, it is also far from the most complicated method in this class. The first step in this method is to prompt the FileStructure instance to reconstruct the file system. Then, the path to the current puzzle file and the puzzle depth are used to determine what the starting location of the player should be in the real file system, with the FolderOpener instance being informed of the result. After that, the number of mistakes made in the current level is reset to zero and the introductory dialogue for the level is played. This ensures that the level is set up properly for the player to be able to interact with it fully.

With that, the detailed discussion of the vital modules in the game is complete. This should provide a more detailed and complete understanding of how

the game creates levels and puzzles, as well as how the player progresses from the start of the game to the end. However, while this concludes the modules present in the game, there is one final piece of software that has not yet been discussed.

CHAPTER 5

EVALUATION

5.1 Experiment

Since this system is intended to teach middle and high school students, it makes sense to involve that demographic in the testing process. Towards this end, students were recruited from Albuquerque Academy in Albuquerque, NM, as well as Loving High School in Loving, NM. During the testing process, each student was asked to play through the tutorial program to ensure they have some starting knowledge, and then also asked to play through the actual game. Due to the relatively low number of responses that were collected, this experiment does not involve the use of a control group. All experiment procedures were done according to the guidance of the IRB of NMT¹. For these students, it is hypothesized that students will gain a better understanding of how to use the digital file system after playing the game. Additionally, there are a number of hypotheses surrounding possible correlations between different aspects of student behavior:

1. H1: A positive correlation will exist between improvement of understanding and participation. This may indicate whether or not simply playing the game leads to knowledge gain.
2. H2: A positive correlation will exist between improvement of understanding and in-game success. This may indicate whether or not doing well in the game leads to knowledge gain.
3. H3: A positive correlation will exist between perceived usability and participation. This may indicate whether or not the game is properly engaging students.
4. H4: A positive correlation will exist between perceived usability and in-game success. This may indicate whether or not students who were more successful in-game found it to be more engaging.
5. H5: A positive correlation will exist between improvement of understanding and perceived usability. This may indicate whether or not making the game more engaging and easier to use might increase knowledge gain.

¹IRB #2024.07.001

5.1.1 Procedure

With the different tests and surveys, as well as needing students to play through the tutorial and the game, it was not possible to complete testing in one day. Due to this, the tests were conducted across two separate days, with the students completing different steps on each. On the first day, students were introduced to the project and the researcher conducting the test, to provide context for what was going to take place. After that, students were asked to play through the tutorial program. Overall, the procedure of the first day was simple and short, with the second day being more substantial. The events of this day alongside the time allocated for them can be found in table 5.1.

At the beginning of the second day of testing, students were given a brief reminder about the project as well as a summary of what would be happening that day. From there, students were given a short understanding quiz, acting as a pretest. Each student was given roughly ten minutes to complete this quiz. Once all quizzes were returned to the researcher, students were free to begin playing the game. After logging in, each student was given time to simply play the game, proceeding through the introductory sequence into the generated levels, and finally to the outro sequence. The student's ability to receive help was restricted to ensure that the vast majority of instruction was coming from the game itself. The only situations where students received aid was regarding how to delete, create, and move files via the file system interface, since those concepts were required to complete the game but had been overlooked in the tutorial. Overall, students were given twenty minutes to play the game, with students who did not complete it in that time being told to close the game early. Once the students were done with the game, they were given the final round of quizzes and surveys.

First and foremost, students were given ten minutes to complete a second understanding quiz, acting as a posttest. Next, students were given two surveys to fill out during the remaining class time, those being the demographic survey and the usability survey. The demographic survey was designed to collect basic demographic information that may be relevant to student performance in the game, as well as knowledge gain. It was emphasized to each student that all of the questions in the survey were optional, and that they were free to not answer any question that they were not comfortable with. The usability survey was designed to collect certain sentiments the students had regarding the game, chiefly how easy to use they felt it was and how useful it seemed to be. After this, students were thanked for their participation and the experiment was concluded. The different activities for the second day of testing can be found in table 5.1. All documents used in this experiment can be found in the appendix at the end of this paper.

	Activity	Duration
Day 1	Introducing project	5 minutes
	Playing tutorial	40 minutes
	Distributing and explaining data collection consent forms	5 minutes
Day 2	Reintroduce project and collect consent forms	5 minutes
	Pretest	10 minutes
	Play game	20 minutes
	Posttest	10 minutes
	Surveys and thanking participants	5 minutes

Table 5.1: Activities for both days of testing.

5.2 Data Remediation

With the abundance of data collected from gameplay and the documents students were given, it was necessary to clean this data of anything invalid. The first step of this cleaning process was to make the data more digestible, allowing any potential errors to be seen. This involved placing all data collected (both from documents and gameplay) into spreadsheets. For any students who provided incomplete data (such as those who did not complete the usability survey), their data was discarded. After this, it was noted that some of the entries in the ID registry seemed to be incorrect. More specifically, certain IDs had been duplicated, indicating that a few students had gotten confused and replayed the game after they finished it initially.

Fortunately, these students still had a fully valid completion of the game before they started it again. To fix this issue and make the data from those students usable, a Python program was written that walked through the sequence data and ensured that all data was continuous and sequential. In any place where the data was not in this state, the program flagged it and it was manually corrected to only include that student's initial completion of the game. At the end of this data remediation process, a total of 31 students were found to have valid data that could be analyzed and processed.

5.3 Data Synthesis

The first step in the data synthesis process is to gain a better understanding of what demographics were represented in the testing population. As previously stated, the demographic survey that was given to students was designed to gather information on certain demographic categories that may be important to the analysis of the data. Firstly, since it is known that systems can sometimes advantage one gender group over another, information on participant gender was

collected. Next, since the aim of this project is to educate students on computer literacy, it seemed sensible to collect information on participant computer use. Towards this end, participants were polled regarding whether or not they have computer access at home as well as how many hours they spend on the computer per week. Additionally, since this is a game-based system, participants were also asked how many educational games they had previously played. Next, since the two schools involved were at different grade levels and located within different regions, participants were grouped according to these metrics. More specifically, participants were grouped into middle school or high school depending on their response to the demographic survey question. For the region, those from the Loving, NM school were marked as rural while those from the Albuquerque, NM school were marked as urban. These are the demographic groups that were decided to be important to this study. Details on group compositions can be found in table 5.2 below.

Category Split	Demographic	Count	Percentage
Gender	Female	12	39%
	Male	17	55%
	Prefer not to say	2	6%
Computer Access	Yes	27	87%
	No	3	10%
Previous Educational Games	0-1	9	29%
	2-3	10	32%
	4-5	6	19%
	6+	5	16%
Weekly Computer Hours	0-2	2	6%
	2-5	10	32%
	5-15	10	32%
	15-30	4	13%
	30+	5	16%
Grade	Middle School	16	52%
	High School	14	45%
Region	Urban	15	48%
	Rural	16	52%
Overall	N/A	31	100%

Table 5.2: Participant counts by each demographic.

While the data left at the end of the data remediation process was cleaned of anything invalid, it was still raw data that was difficult to use. To remedy this issue, the data needed to be properly analyzed into something that is human-readable. The first step of this was completed in the spreadsheets mentioned

earlier, with some very basic information being collected from the raw data. The data synthesized here included:

- The total scores for the pretest and posttest, synthesized from the scores on each individual question.
- The total scores for each question topic of the pretest and posttest, synthesized from the scores of the contributing questions.
- The understanding improvement experienced by students, synthesized by subtracting their pretest scores from their posttest scores.
- The total scores for the usability survey, including the overall score, the ease of use score, and the usefulness score.
- Usability keywords, synthesized by reading through the open-ended responses to the usability survey and noting any common keywords.

For the understanding improvement metric, it was also necessary to verify that the data follows a normal distribution, as required for some of the analysis performed later on. Fortunately, figure 5.1 indicates that this is the case for the most part. As can be seen, the difference between the pretest and posttest scores roughly follows a normal distribution, allowing for t-tests and ANOVA tests to be completed. This is further shown by a Kolmogorov-Smirnov, as the p-value of 0.23 that this test provides is not sufficient to reject the null hypothesis of the understanding improvement following a normal distribution.

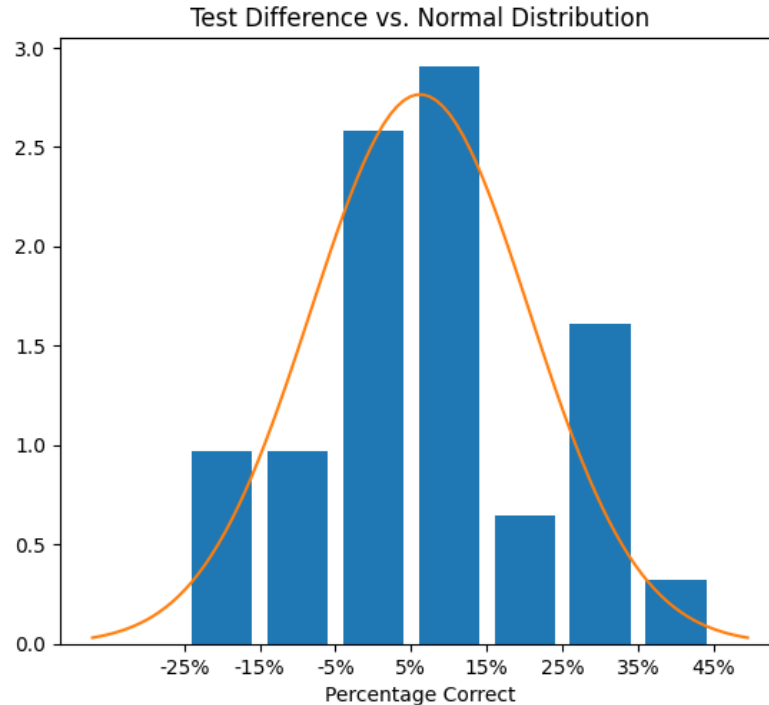


Figure 5.1: A histogram of knowledge gain compared to the normal distribution. Note that the height of the bars has been adjusted such that the area of the histogram is the same as the area of the normal distribution.

With this more simple synthesis completed, further information was calculated. Firstly, the means and standard deviations for the different attributes of the pretest and posttest were calculated. More specifically, the mean and standard deviation was calculated for the total scores of each test, as well as each question category:

- Navigation, the ability of the student to navigate from one location to another.
- Reorganization, the ability of the student to reorganize parts of the file system into a more appropriate structure.
- Matching, the ability of the student to match files to the locations that should contain them.
- New Folder, the ability of the student to determine what folder should be created to contain a disorganized file.

This information is already very useful, but fails to capture the whole situation. Since the goal of this project is to create a system capable of teaching students about the file system, it is necessary to understand how students actually improved between the two understanding quizzes. For this reason, the mean

and standard deviation of the difference between pretest and posttest scores was calculated for each student. It should be noted that this is not a calculation of the differences between the two averages, it is specifically an average of the difference in test results for each student. The mean and standard deviation was also calculated for the improvement in every question category. Overall, these statistics should provide a much more useful insight into how students performed on both tests. The results of these mean and standard deviation calculations can be found in table 5.3.

	Pretest		Posttest		Difference	
	Average	STD	Average	STD	Average	STD
Navigation (Max=2)	1.16	0.78	1.23	0.8	0.06	0.85
Reorganization (Max=3)	2.58	0.67	2.77	0.5	0.10	0.83
Matching (Max=4)	3.81	0.79	3.81	0.79	0.0	0.52
New Folder (Max=3)	2.13	0.79	2.71	0.76	0.58	0.89
Total (Max=12)	9.68	1.7	10.42	1.48	0.74	1.73

Table 5.3: Averages and STDs of student performance for each test metric.

After this, some final data synthesis was performed in order to provide insights into some more specific topics. Firstly, synthesis was performed on the puzzle sequence data of each student. For each student, a program extracted the maximum difficulty puzzle they had successfully completed, the number of puzzles they had completed successfully, and the number of puzzles that they had failed. From this, the total number of puzzles that each student encountered was found, as well what proportion of those puzzles were completed successfully. All of this synthesized data was then stored alongside the raw spreadsheet data in a large Pandas dataframe. This dataframe was also used to generate smaller dataframes containing students with similar demographic information (for example, a dataframe with all middle school participants). With this, the data synthesis is complete, and it is possible to move on to the actual analysis.

5.4 Demographic Analysis

The data analysis for this project begins with the demographic analysis. The purpose of this analysis is to determine how students performed on the pretest, posttest, and actual game based on their demographic differences. In this process, qualitative analysis was performed via histograms, alongside the quantitative metrics of t-tests and ANOVA tests. The histograms were created using the pyplot module of the matplotlib Python package, while the quantitative tests were performed with the stats module of the scipy package. This should allow for a better understanding of the performance difference between different demographic groups.

While the focus of this section is the demographic analysis, it is important to obtain a baseline behavior for the overall group. Towards this end, histograms were generated using the data of the overall group, showing how they performed on both understanding quizzes, how their scores changed between those quizzes, and how they performed in the game itself. These histograms can be found in figure 5.2 below.

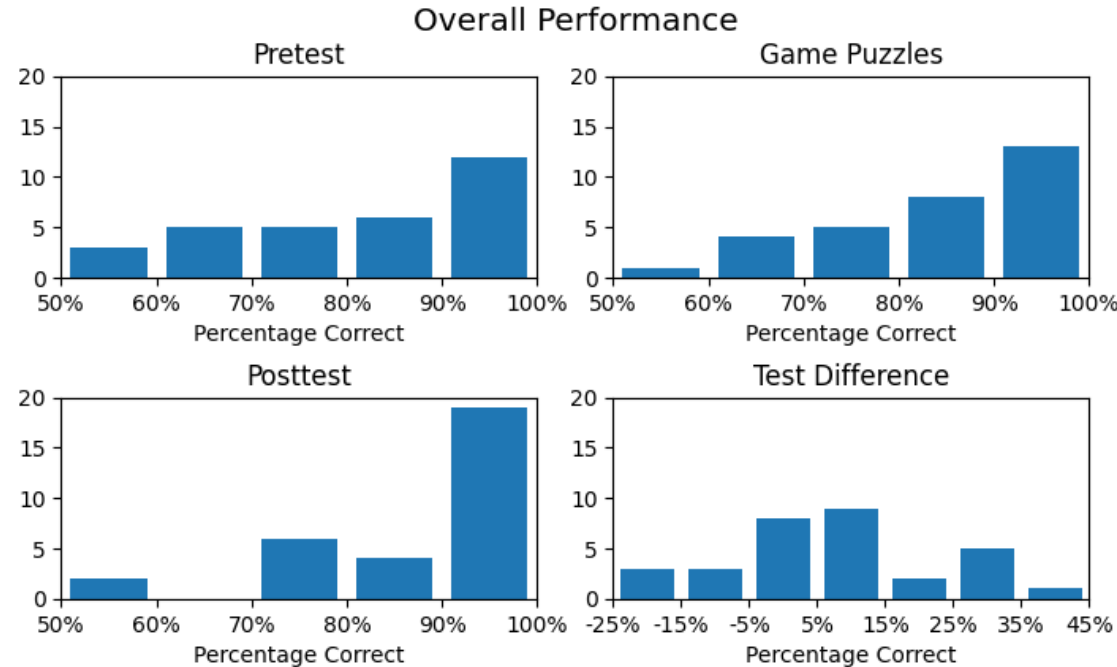


Figure 5.2: Histogram of pretest, posttest, and game performance.

After this, histograms were generated showing what the maximum difficulty each student completed was, as well as how many levels they encountered. This was done to provide a more detailed view of the game performance of the overall group. The maximum difficulty completed provides a better understanding of how far into the game students are progressing, while the levels encountered indicates how many students are finishing early or being cut off due to time. The visualization of this data can be found in figure 5.3 below. Finally, a scatterplot was created from the maximum difficulty completed and levels encountered for the overall group to see if there is a visible trend between the two. This scatterplot can be found in figure 5.4. With this analysis of the overall group done, it is time to move into analyzing the different demographic groups. It should be noted that, due to the somewhat less informative nature of the last two figures in the overall group, only the first figure will be generated for each of the demographic groups.

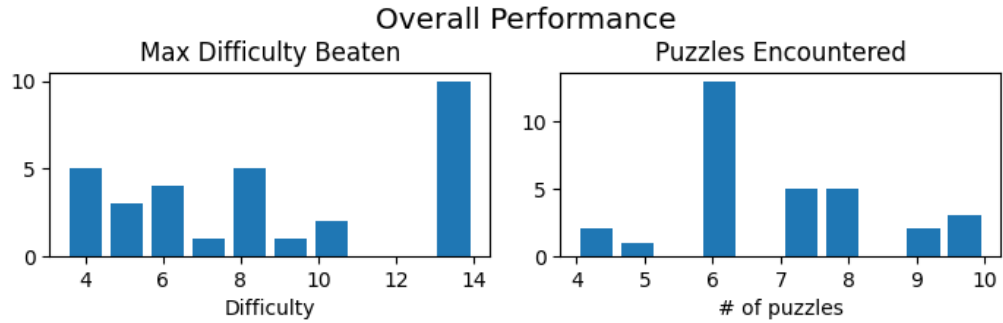


Figure 5.3: Histogram of maximum difficulty beaten and number of puzzles encountered.

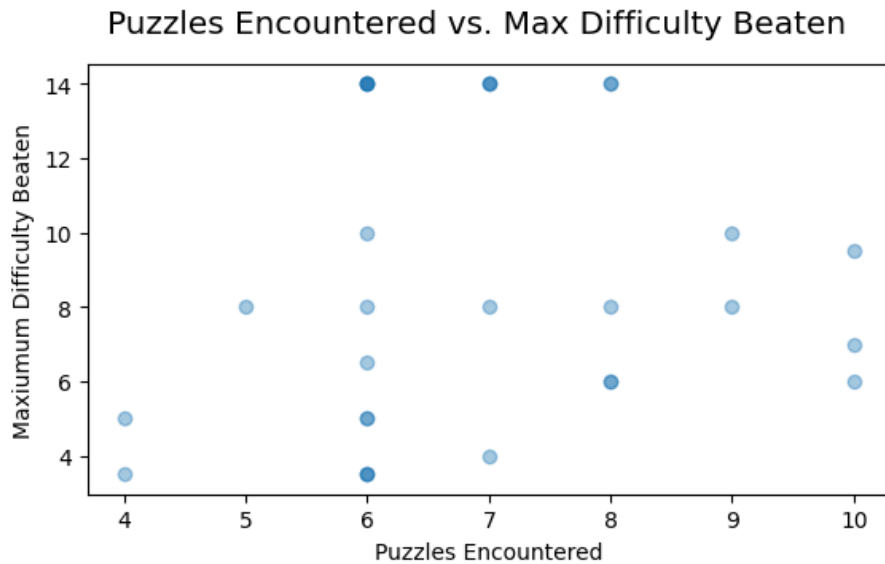


Figure 5.4: Scatterplot of maximum difficulty beaten vs. number of puzzles encountered.

Beginning the demographic analysis, the small sample sizes indicated in table 5.2 earlier should be noted. Due to this sample size, it is unlikely for any of the quantitative tests to produce statistically significant results, although they will still be performed just in case. The goal of this analysis is to determine how different demographics experienced the game, as well as if there are any differences between the groups. For each demographic group that was analyzed, histograms were generated to see if there were any visible trends for pretest performance, posttest performance, test performance difference, and the proportion of puzzles passed. After this, t-tests were performed on each group to see if any meaningful difference existed between them. For demographics that were split into more than two groups, these t-tests were performed by combining groups into roughly

equal halves. Additionally, for the gender t-test, the “prefer not to say” group was excluded due to the very low sample size. For each demographic with more than two groups, an ANOVA test was also performed to indicate whether any meaningful differences existed between the groups. The histograms generated during this process can be seen in figures 5.5-5.10 below, while the quantitative results are in tables 5.4-5.12 below. With the demographic analysis concluded, the remainder of the analysis may take place.

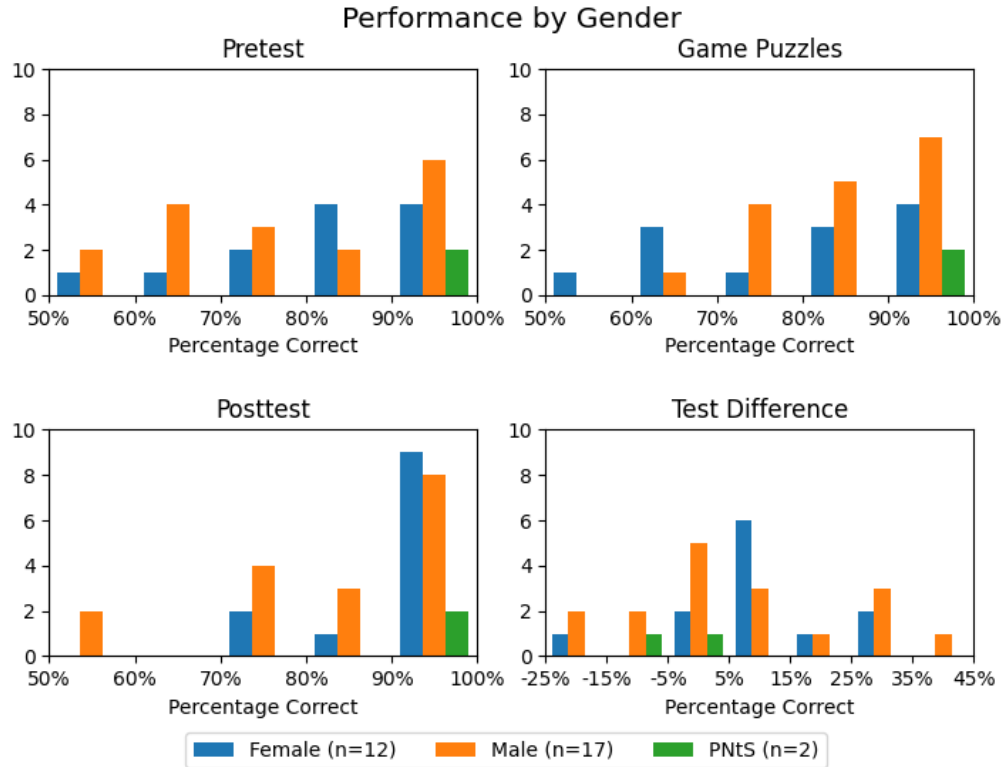


Figure 5.5: Histograms of performance by gender.

Gender	Metric	Avg	SD	d.f.	T-Statistic	P-Value
Male (n=17)	Test Difference	6%	17%	26.95	-0.47	0.64
Female (n=12)	Test Difference	8%	11%			

Table 5.4: T-Test comparing score improvement by gender. H_a is $\neq$.

Gender	Metric	Avg	SD	F-Statistic	P-Value
Male (n=17)	Test Difference	6%	17%	0.64	0.54
Female (n=12)	Test Difference	8%	11%		
Prefer Not to Say (n=2)	Test Difference	-4%	6%		

Table 5.5: ANOVA test comparing score improvement by gender.

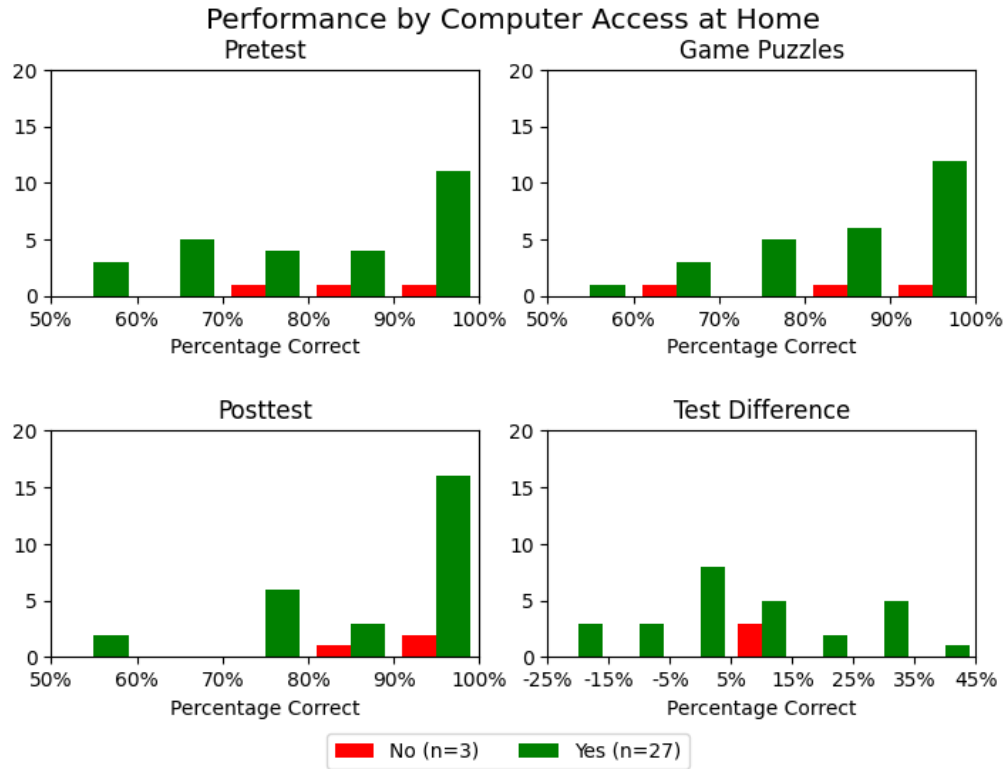


Figure 5.6: Histograms of performance by computer access.

Computer Access	Metric	Avg	SD	d.f.	T-Statistic	P-Value
Yes (n=27)	Test Difference	6%	15%	26.0	0.83	0.41
No (n=3)	Test Difference	8%	0%			

Table 5.6: T-Test comparing score improvement by computer access. H_a is $\neq$.

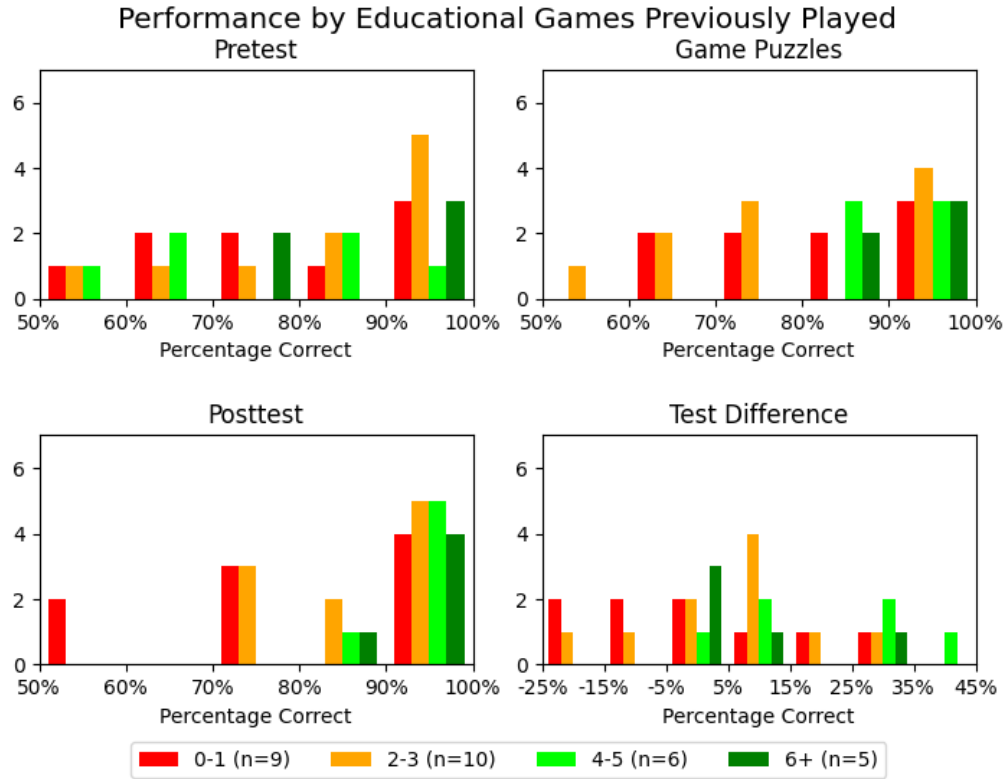


Figure 5.7: Histograms of performance by educational games previously played.

Previous Games	Metric	Avg	SD	d.f.	T-Statistic	P-Value
≤ 3 (n=19)	Test Difference	2%	14%	20.67	-2.01	0.058
≥ 4 (n=11)	Test Difference	13%	14%			

Table 5.7: T-Test comparing score improvement by educational games previously played. H_a is $\neq$.

Previous Games	Metric	Avg	SD	F-Statistic	P-Value
0-1	Test Difference	-1%	16%	2.32	0.10
2-3	Test Difference	5%	12%		
4-5	Test Difference	18%	15%		
6+	Test Difference	7%	11%		

Table 5.8: ANOVA test comparing score improvement by educational games previously played.

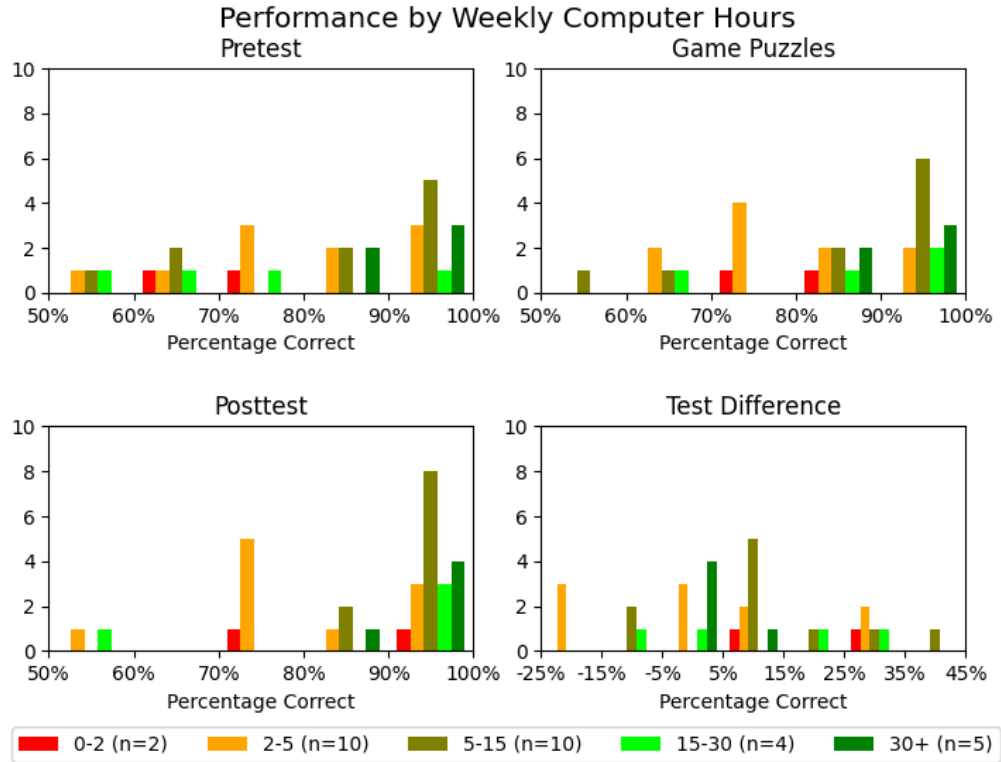


Figure 5.8: Histograms of performance by weekly computer hours.

Weekly Hours	Metric	Avg	SD	d.f.	T-Statistic	P-Value
< 5 (n=12)	Test Difference	3%	17%	19.03	-0.78	0.45
> 5 (n=19)	Test Difference	8%	13%			

Table 5.9: T-Test comparing score improvement by weekly computer hours. H_a is $\neq$.

Weekly Hours	Metric	Avg	SD	F-Statistic	P-Value
0-2	Test Difference	17%	12%	1.01	0.42
2-5	Test Difference	1%	17%		
5-15	Test Difference	11%	15%		
15-30	Test Difference	8%	15%		
30+	Test Difference	2%	4%		

Table 5.10: ANOVA test comparing score improvement by weekly computer hours.

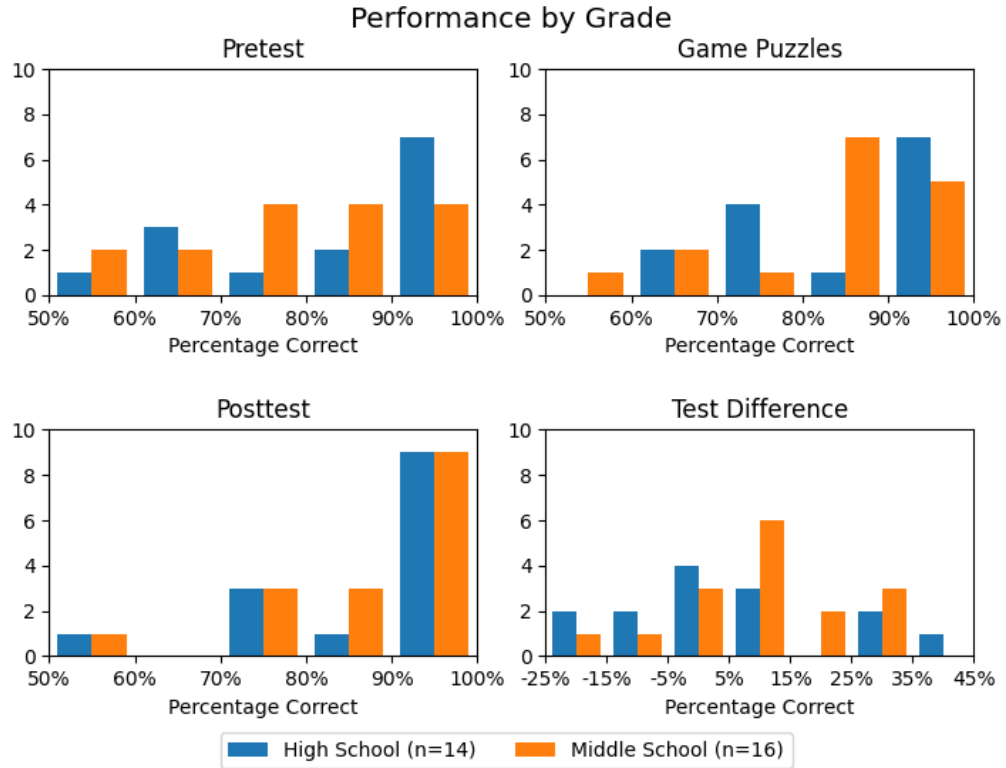


Figure 5.9: Histograms of performance by grade.

Grade	Metric	Avg	SD	d.f.	T-Statistic	P-Value
High School (n=14)	Test Difference	4%	18%	22.30	-0.75	0.46
Middle School (n=16)	Test Difference	8%	12%			

Table 5.11: T-Test comparing score improvement by grade. H_a is $\neq$.

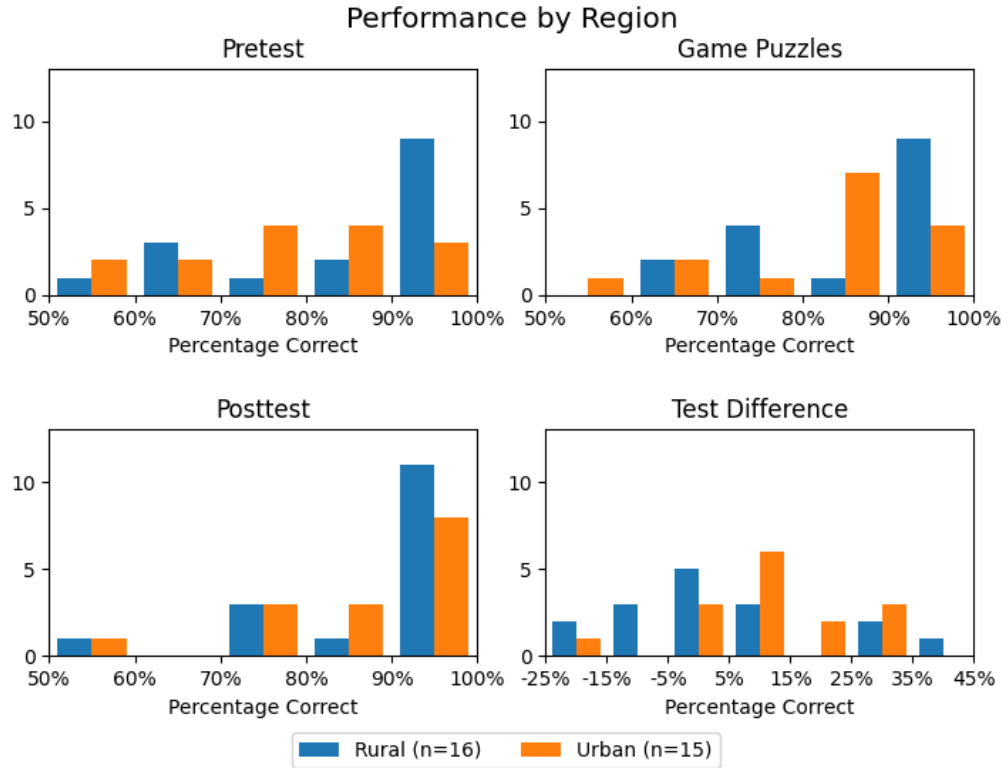


Figure 5.10: Histograms of performance by region.

Grade	Metric	Avg	SD	d.f.	T-Statistic	P-Value
Rural (n=16)	Test Difference	9%	11%	26.52	1.24	0.22
Urban (n=15)	Test Difference	3%	17%			

Table 5.12: T-Test comparing score improvement by region. H_a is $\neq$.

5.5 Further Quantitative Analysis

5.5.1 Overall Correlations

The first step to be taken in the remaining analysis is determining if any meaningful correlations exist between different aspects of the collected data. While some information could be gained simply by checking the correlations between every metric, the analysis done here was more targeted than that. More specifically, the correlation analysis performed was specifically done to answer the correlation hypotheses outlined in section 5.1 earlier. While these hypotheses seem to be important to answer to understand the performance of the system, an aspect of them needs to be addressed first. Specifically, numerous hypotheses refer

to participation and system success, but neither of those are metrics that can be directly measured. In order to evaluate these hypotheses, some aspect of the collected data must be used as a metric of those concepts.

Towards that end, the number of levels encountered is used as a stand-in for participation. This metric does have notable flaws, such as the fact that it can be seen as an inverse to system success since students who do well and finish early will be seen as having less participation. However, other metrics that could be used have similar or more drastic issues, making the number of levels encountered the best approximation of participation available. Fortunately, system success has a much more appropriate metric available. To represent this concept, the proportion of puzzles that student's passed is used. This metric represents how often students failed puzzles versus how often they succeeded, which seems very appropriate for this purpose. While this metric does have some issues (such as not accounting for students who only do easy puzzles and then stop), it should still be very usable. Finally, it is necessary to use some metric to represent knowledge gain. As previously discussed, the pretest and posttest were made explicitly for this purpose, so the difference between the overall scores of these tests will be used to represent knowledge gain. With all of these metrics and the hypotheses to test established, it is possible to move into the actual correlations.

Overall, the process of calculating the different correlations was fairly simple. As previously mentioned, all of the data collected in this project is stored in a large Pandas dataframe, making it trivial to write Python code that processes it. A method from the stats module of the scipy package was used to perform Pearson correlations between all of the relevant metrics, with the results of this being found in table 5.13. While this provides a quantitative analysis of any correlations that may exist, a qualitative view would also be beneficial. Towards this end, scatterplots for each pair of target metrics were also generated, being found in figures 5.11-5.13 below. This analysis provides a solid understanding of what correlations may or may not exist in the game. However, this is not the last aspect of the analysis left to discuss.

	Puzzles Encountered	Puzzles Passed	Test Improvement	Usability
Usability	-0.04/0.83	+0.37/0.04	+0.12/0.51	+1.0
Test Improvement	-0.00/0.96	+0.11/0.57	+1.0	
Puzzles Passed	-0.52/0.002	+1.0		
Puzzles Encountered	+1.0			

Table 5.13: Correlations among the different hypothesis variables. A bolded value indicates a statistically significant result. Cells follow the format "correlation/p-value".

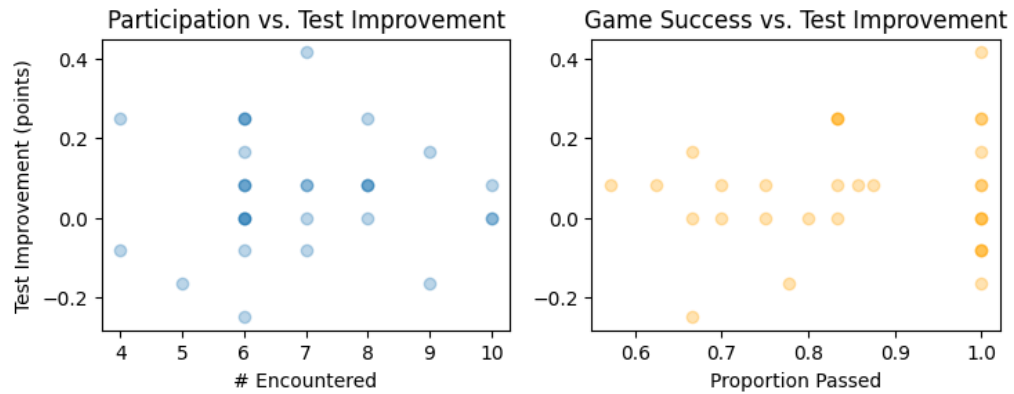


Figure 5.11: Test improvement vs. puzzles encountered and proportion beaten.

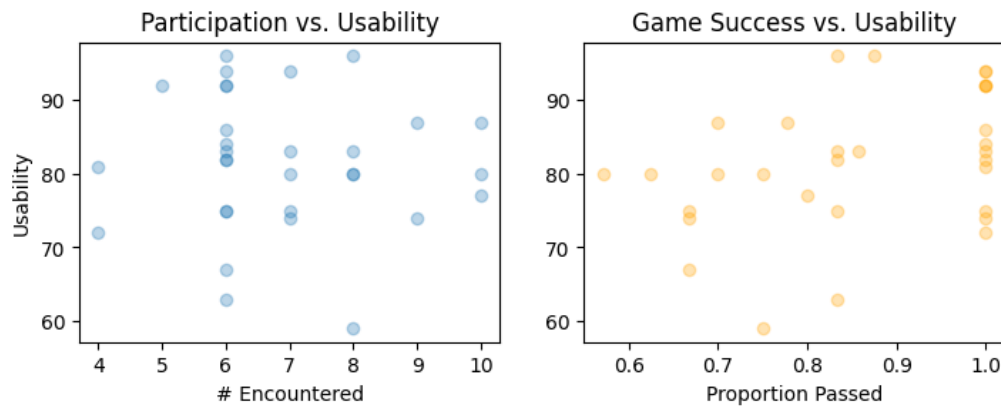


Figure 5.12: Usability vs. puzzles encountered and proportion beaten.

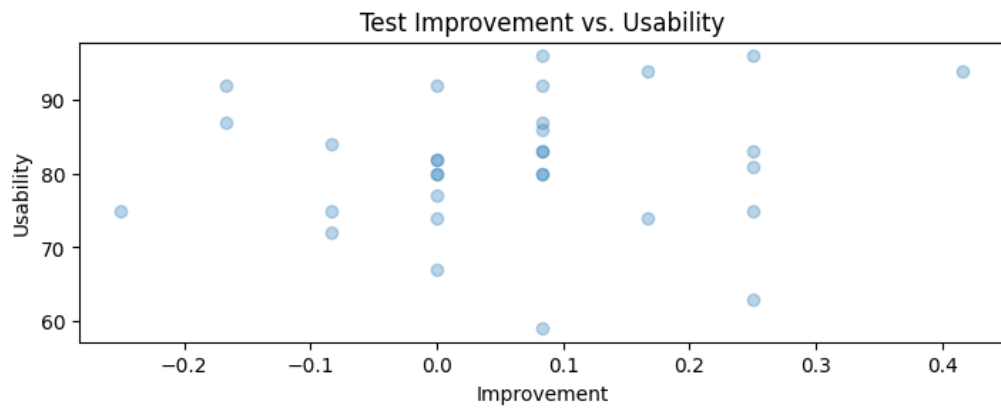


Figure 5.13: Usability vs. test improvement.

5.5.2 Overall T-Tests

While the correlation analysis is useful, performing more t-tests would provide a more proper understanding of the score differences between the pretest and the posttest. Importantly, this understanding is key to supporting the primary hypothesis presented in section 5.1, that the game improves student understanding of the digital file system. Rather than performing these tests manually, the stats of the module of the scipy package is used once again. For these tests, the only null hypothesis used is that there is no difference between the average scores of the pretest and the average scores of the posttest. Using a t-test with this null hypothesis, it is possible to determine whether or not the game meaningfully improved student understanding of the digital file system. With this baseline established, the t-tests can be performed.

The t-test was performed using a method of the stats module in the scipy package, determining if the difference between the pretest scores and the posttest scores is statistically significant. Specifically, the test performed was a two-tailed paired t-test comparing the total scores of the pretest with the total scores of the posttest. To achieve this, the `ttest_rel` method was used, with the results of the test being found in table 5.14. This provides meaningful results which will be discussed later, meaning that it is now time to move on to the remaining aspects of the data analysis.

Test	Metric	Avg	SD	d.f.	T-Statistic	P-Value
Pretest (n=31)	% Correct	81%	14%	30	-2.39	0.02
Posttest (n=31)	% Correct	87%	12%			

Table 5.14: T-Test comparing pretest and posttest scores. H_a is $\neq$.

5.5.3 Split Correlations

Due to the highly variable nature of the pretest scores, analysis was done to see if those who scored higher on the pretest had different trends than those who scored lower. Towards this end, the overall group was split in two and the correlation analysis was repeated. In the overall group, the median pretest score was found to be ten points. Once this was found, the group was split in two by placing everyone who scored at or below the median into the low scoring group, while everyone who scored above the median was placed into the high scoring group. It should be noted that the median was used due to the skewed nature of the pretest scores. In the end, this created a low scoring group consisting of 19 participants and a high scoring group consisting of 12 participants. While a more even split would be ideal, too many students scored exactly the same as the median value for this to be possible. With these two groups established, the split correlation analysis becomes possible.

For finding the correlations of the two groups, the same analysis was done as for the overall group. This provides the correlations as seen in the tables below, with the high scoring group being represented by table 5.15 while the low scoring group is represented by table 5.16. Additionally, the same scatterplots were generated as the overall group, with the high scorers being represented by figures 5.14-5.16 and the low scorers being represented by 5.17-5.19.

	Puzzles Encountered	Puzzles Passed	Test Improvement	Usability
Usability	+0.24/0.45	+0.24/0.45	+0.09/0.78	+1.0
Test Improvement	+0.11/0.74	-0.27/0.40	+1.0	
Puzzles Passed	-0.40/0.20	+1.0		
Puzzles Encountered	+1.0			

Table 5.15: Correlations among the different hypothesis variables in the high-scoring group. A bolded value indicates a statistically significant result. Cells follow the format "correlation/p-value".

	Puzzles Encountered	Puzzles Passed	Test Improvement	Usability
Usability	-0.13/0.59	+0.43/0.06	+0.26/0.29	+1.0
Test Improvement	-0.24/0.32	+0.58/0.01	+1.0	
Puzzles Passed	-0.54/0.02	+1.0		
Puzzles Encountered	+1.0			

Table 5.16: Correlations among the different hypothesis variables in the low-scoring group. A bolded value indicates a statistically significant result. Cells follow the format "correlation/p-value".

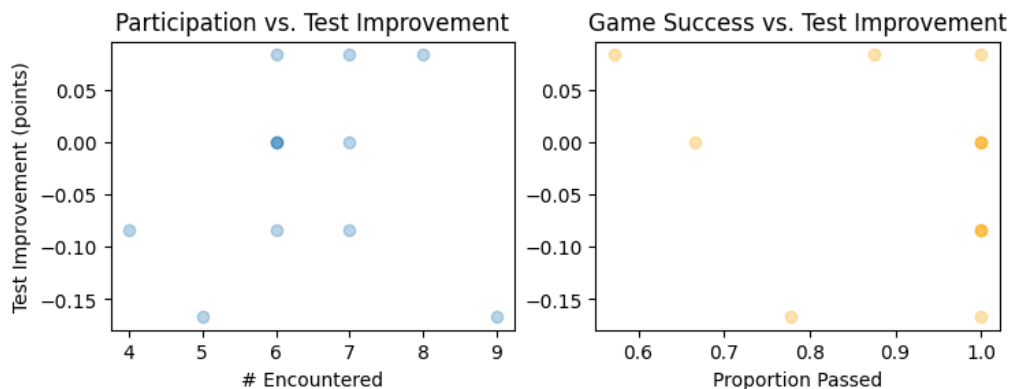


Figure 5.14: Test improvement vs. game metrics in high scorers.

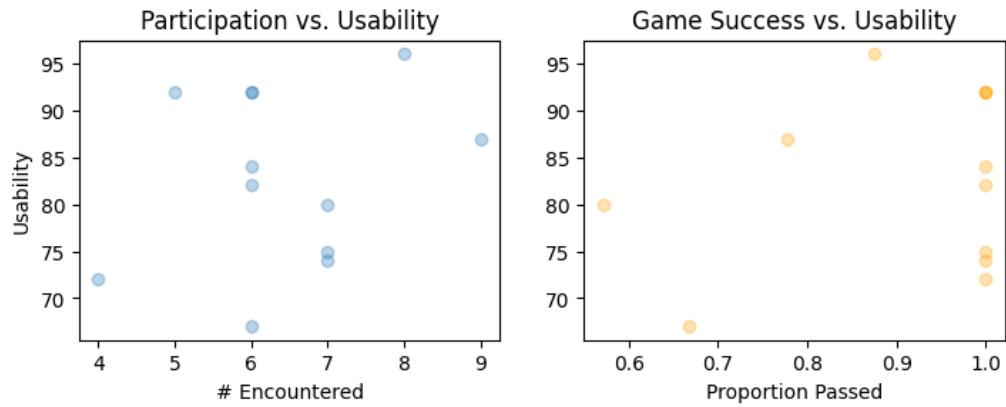


Figure 5.15: Usability vs. game metrics in high scorers.

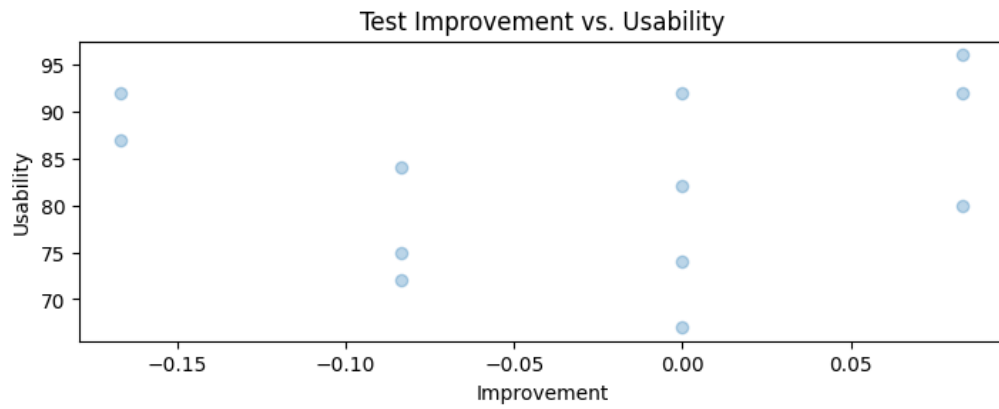


Figure 5.16: Usability vs. test improvement in high scorers.

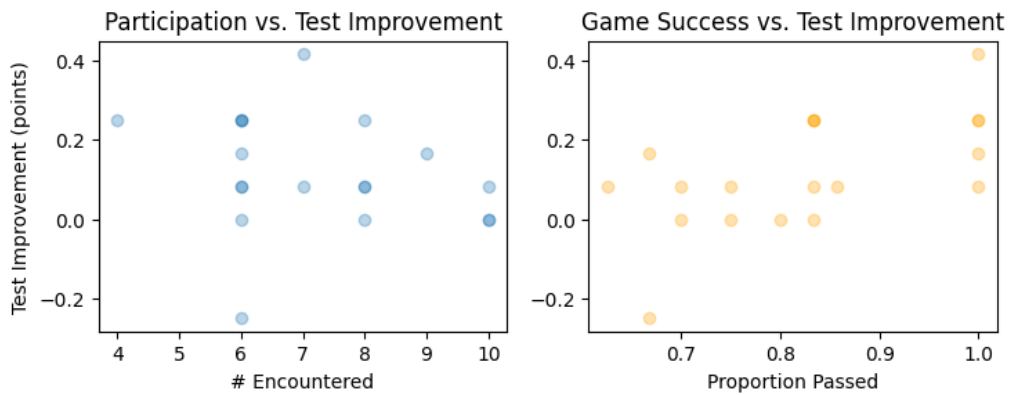


Figure 5.17: Test improvement vs. game metrics in low scorers.

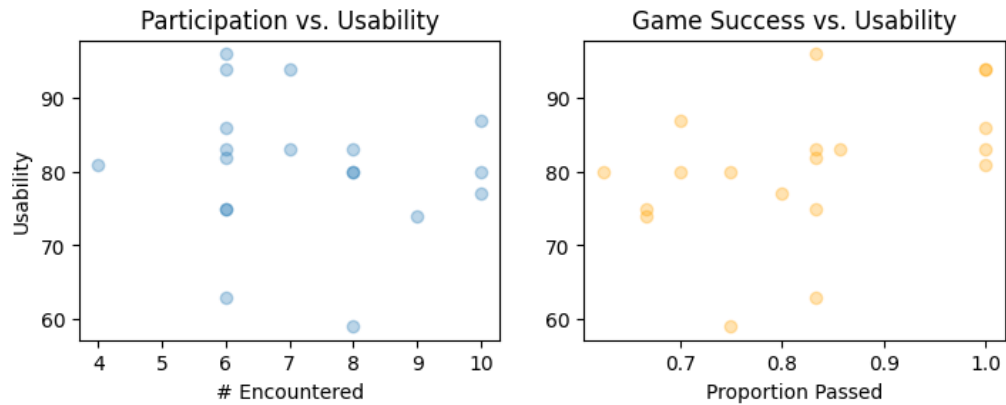


Figure 5.18: Usability vs. game metrics in low scorers.

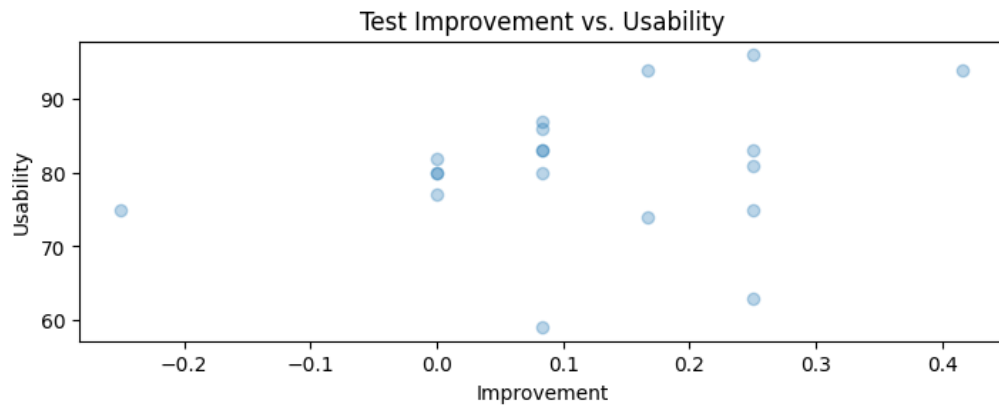


Figure 5.19: Usability vs. test improvement in low scorers.

5.5.4 Split T-Tests

Since some difference was noted in the correlations of the two groups, the t-test from the overall group was repeated for the split groups as well. Similarly to the correlations, these t-tests were performed exactly as they were for the overall group. This process is no more complex than it was before, with the results for the high scoring group being found in table 5.17 and the low scoring group being found in table 5.18.

Test	Metric	Avg	SD	d.f.	T-Statistic	P-Value
Pretest (n=12)	% Correct	94%	4%	11	1.08	0.30
Posttest (n=12)	% Correct	92%	9%			

Table 5.17: T-Test comparing pretest and posttest scores for the high-scoring group. H_a is $\neq$.

Test	Metric	Avg	SD	d.f.	T-Statistic	P-Value
Pretest (n=19)	% Correct	72%	11%	18	-3.56	0.002
Posttest (n=19)	% Correct	84%	13%			

Table 5.18: T-Test comparing pretest and posttest scores for the low-scoring group. H_a is $\neq$.

5.5.5 Comparative T-Tests

Reviewing the results of the split t-tests, more differences between the two groups were noted. Due to this, it was decided that a t-test needed to be done in order to see if one group was benefitting from the game more than the other. Towards this end, an independent t-test was performed using the stats module of the scipy package to compare the score improvement from one group to the other. Due to the low sample size, the two groups were not assumed to have equal variance. Similarly to the previous tests, this was a two-tailed test to see if any difference existed between the groups. The results of this test can be found in table 5.19 below. This concludes the performance analysis, allowing for other aspects of the system to be evaluated.

Group	Metric	Avg	SD	d.f.	T-Statistic	P-Value
High-Scoring (n=12)	Test Improvement	-3%	9%	29	-3.47	0.002
Low-Scoring (n=19)	Test Improvement	12%	15%			

Table 5.19: T-Test comparing score improvement for the two score groups. H_a is $\neq$.

5.6 Sequence Analysis

While reviewing the overall performance of the students is useful, another crucial area of analysis is the trends present as students progress through the

game. This analysis is vital to identifying portions of the game that can be improved or where students have notable difficulty. Additionally, this analysis may even provide information on certain aspects of the game that may be difficult to intuit beforehand. While performing a more complex analysis to obtain quantitative results would be ideal, the low sample size makes such analysis impossible. Due to this, the only analysis done here will be qualitative, being completed via different plots. Despite the lack of quantitative data, this should still give some indication of particularly noteworthy trends.

First and foremost, this analysis involved the creation of multiple histograms to visualize different data trends. What the histograms found in figure 5.20 represent is the frequency of different mistake counts for each level and difficulty. This is intended to highlight locations in the game where students abruptly have more difficulty, which may represent difficulty spikes that may need to be smoothed out. Additionally, these histograms may also indicate locations where students do not experience enough challenge. Overall, these figures can inform the difficulty calculations and game design of future work along this line.

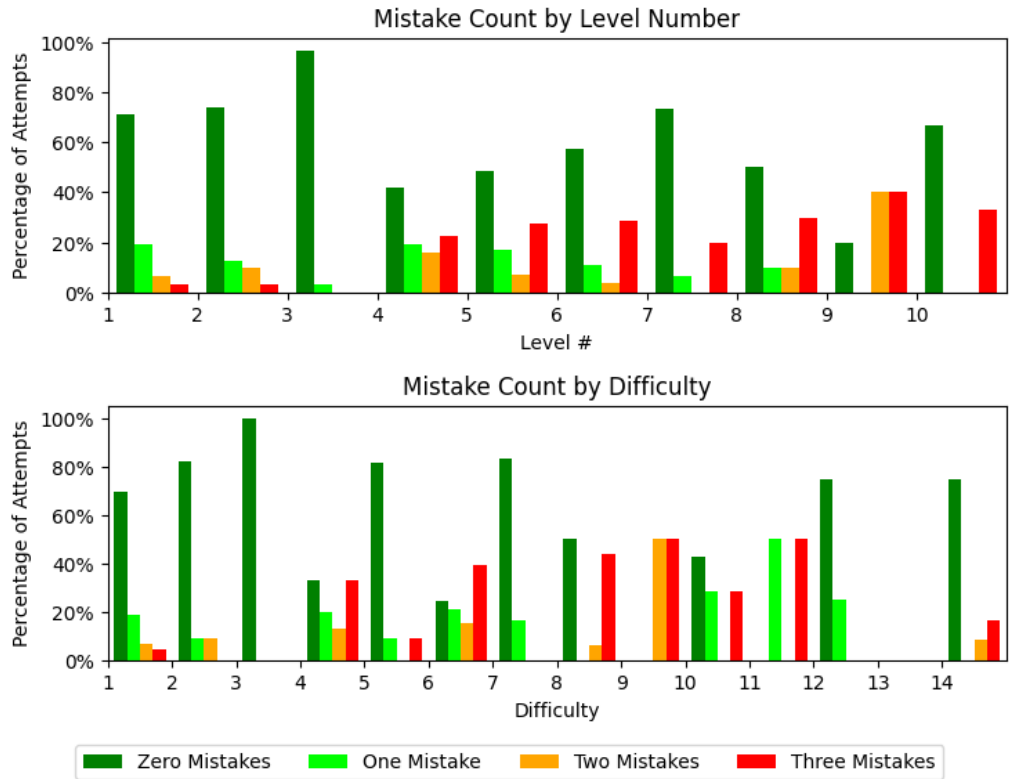


Figure 5.20: Mistake counts by level # and difficulty. Note that heights have been normalized to represent the proportion of attempts per level/difficulty.

To gain a better understanding of how students progress through the game, a line plot was made showing how the difficulty encountered by each student

changed over the course of the game (found in figure 5.21 below). In this plot, each line represents a specific student's path through the game, showing how the difficulty of the puzzles they encountered rose and fell over the course of their experience. This shows the area where they made quick progress (with a rise in difficulty to accompany it) alongside areas where they struggled more (where the difficulty lowers). While this plot is somewhat congested due to the number of students, it does allow these trends to be seen, as well as showing how students exit the game at different points throughout. Overall, this figure provides a different view of those notable moments in the game where students are left without enough or with too much challenge.

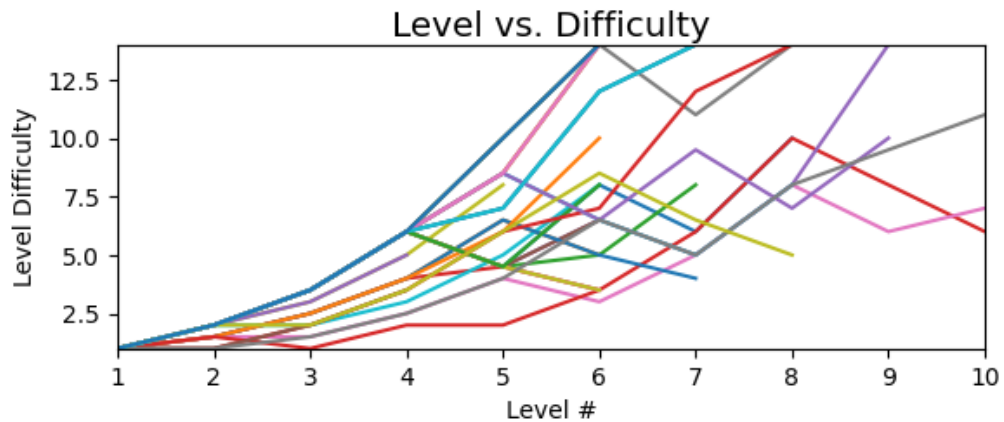


Figure 5.21: Difficulty vs level # for each student.

One final metric that can be used to see how much difficulty students are encountering throughout the game is the time spent on each puzzle. By investigating the amount of time spent on different levels and difficulties, it may be possible to see what points in the game required students to provide more thought. In this, the time plots found in figure 5.22 can provide further insight into what portions of the game may need to be altered in future work. This is also the final step of the sequence analysis, allowing for the final aspect of the analysis to be reviewed.

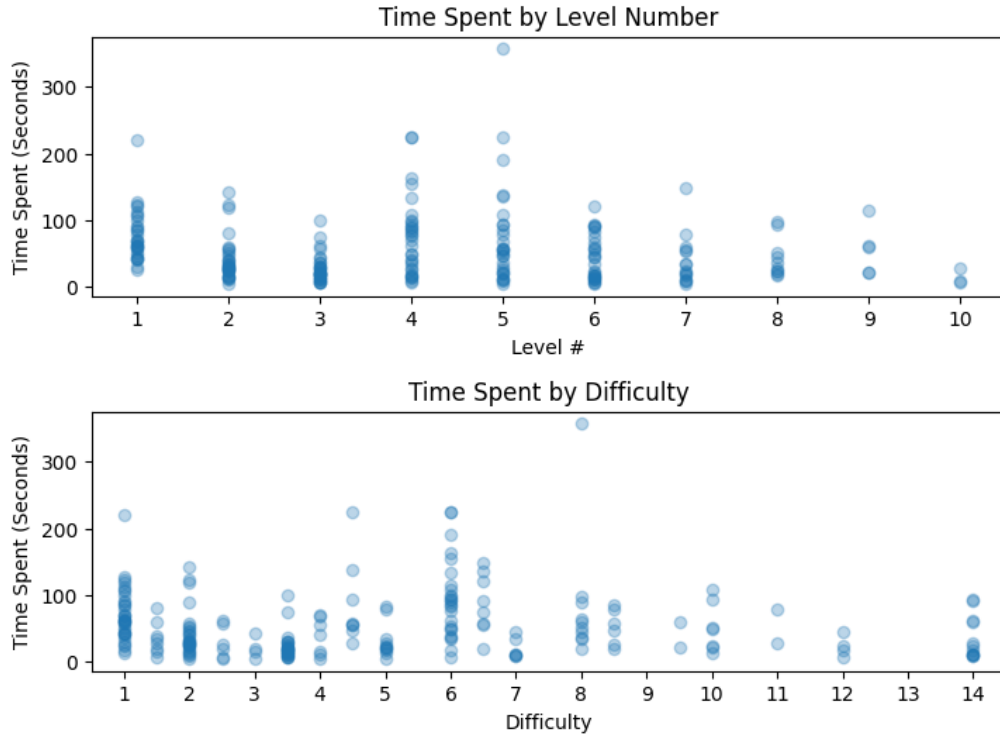


Figure 5.22: Time spent by each student, dependent on level # and difficulty.

5.7 Usability Analysis

While the performance and sequence analysis are both vital elements of this project, another important aspect is the usability of the system itself. The usability of the system is concerned with two primary points, those being the ease with which the system can be used and how useful the system actually is. To determine this, it is necessary to consult the data collected during the usability survey. Evaluating this data should provide crucial insight into how usable the participants found the game to be. Since the concept of usability is not concrete and the sample size of this study is somewhat small, the usability analysis will be constrained to simpler, qualitative methods. With that out of the way, the first portion of the usability data to analyze is the numerical data.

The numerical data, as far as this study is concerned, is the portion of the usability survey where students were asked how much they agree with a given statement. For each question, each student's response is interpreted as a number from one to five, where one is "highly disagree" and five is "highly agree". Since all questions were worded in such a way that a higher score indicates a more usable system, a histogram of overall usability scores was made by averaging a student's scores on each question. This histogram can be found in figure 5.23 below. While these overall scores are useful, they provide little information on

what specific aspects students may have found more or less usable. In the usability survey, questions can be grouped into two categories: those concerned with ease of use, and those concerned with usefulness. Due to this, similar histograms to the overall one were made for these two categories of questions, giving more specific insight into user satisfaction for these two usability components. These histograms can be found in figure 5.24 below.

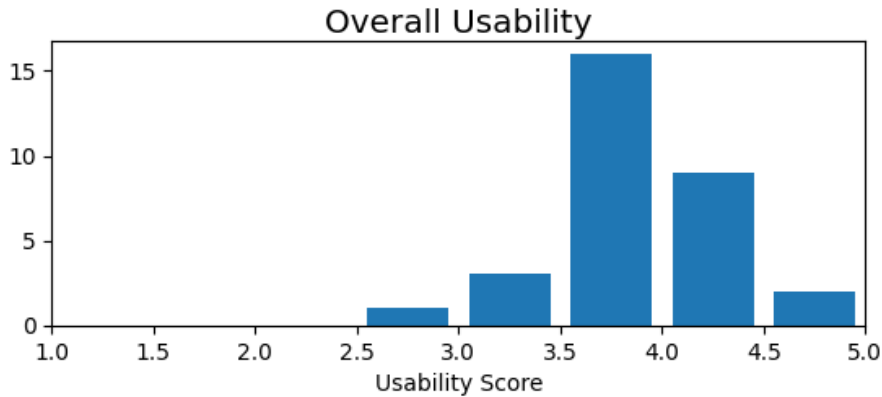


Figure 5.23: Histogram of overall usability scores.

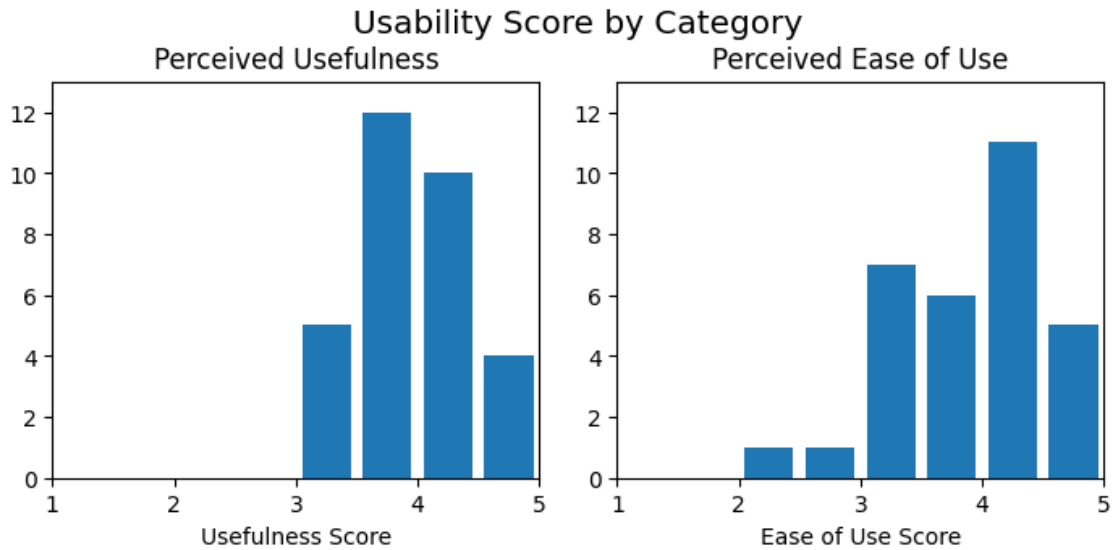


Figure 5.24: Histograms for usability score categories.

The final aspect of usability to be analyzed is the open-ended questions given at the end of the survey. These questions may provide information about issues

that were not accounted for in the remainder of the survey. Due to this, these open-ended questions are vital to consider. Firstly, bar plots showing the frequency of different keywords were created. One plot was made for each question, with the questions having the following points of focus:

- Engagement, what does the user think can be done to make the game more engaging.
- Trouble, what aspects of the game caused the user notable difficulty.
- Improvement, what does the user think can be done to make the game better in general.

Combined, the results from these questions should give a decent indication of what can be done to make the game more usable. The plots for all three of these questions can be found in figures 5.25-5.27 below. It should be noted that “None” indicates that a participant explicitly stated that they had no feedback, while “did not answer” means that they left the question blank.

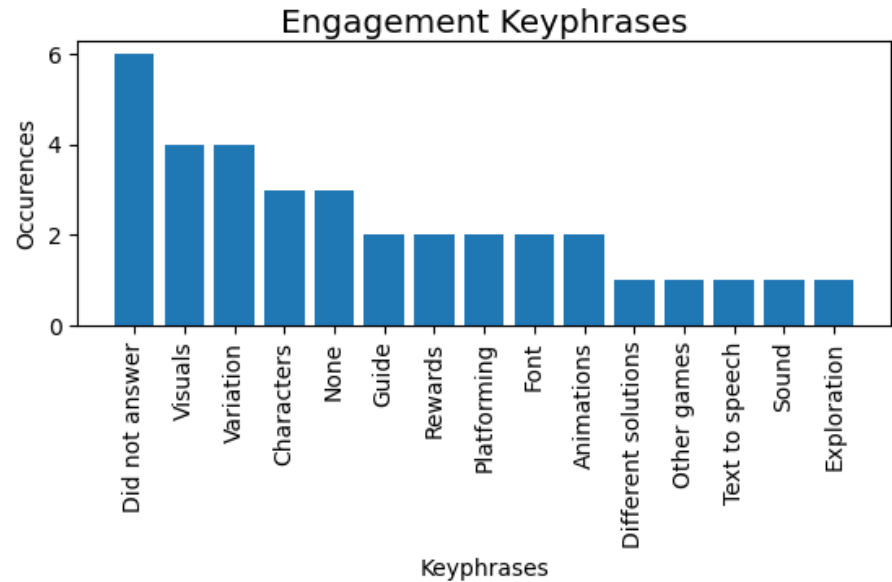


Figure 5.25: # of times each phrase was mentioned regarding engagement.

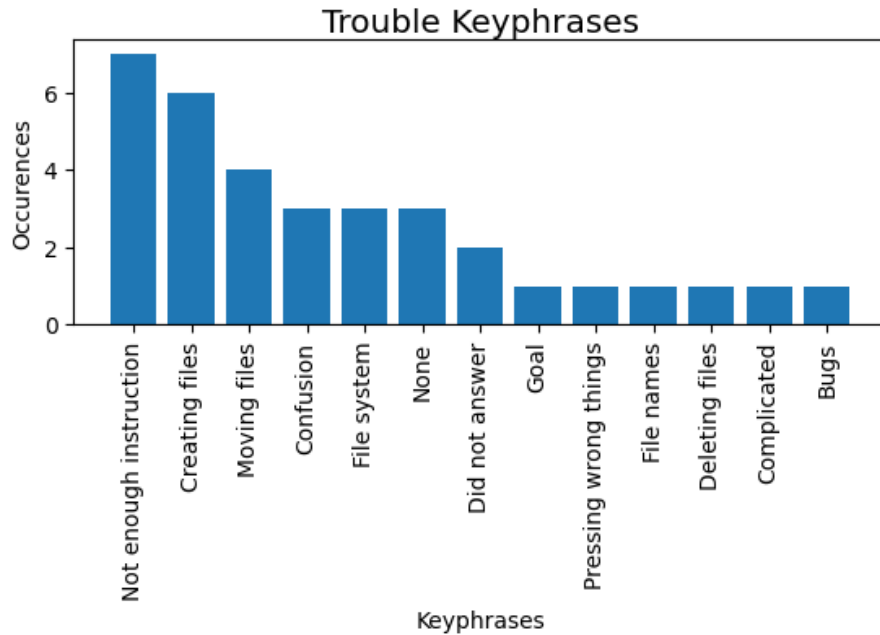


Figure 5.26: # of times each phrase was mentioned regarding trouble in-game.

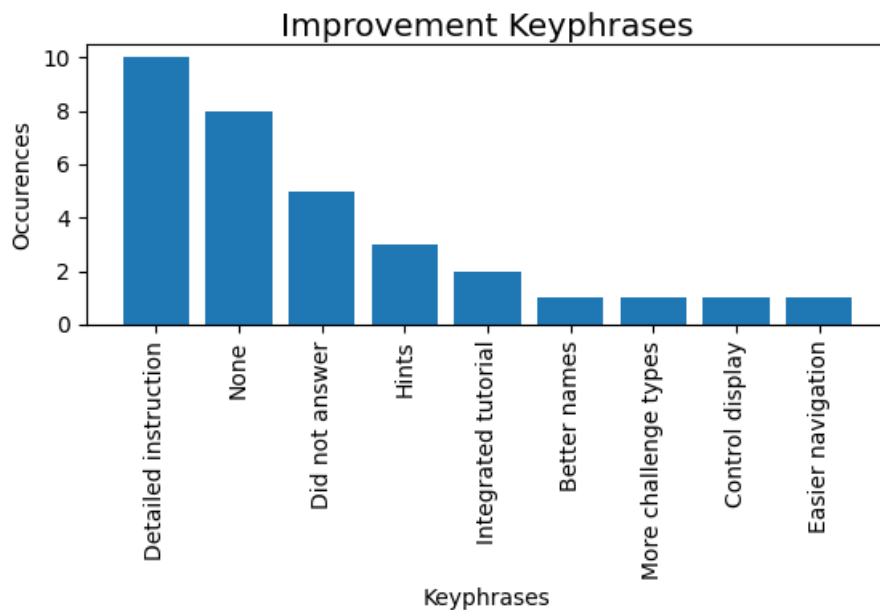


Figure 5.27: # of times each phrase was mentioned regarding improving the game.

To provide a broader view of feedback, keywords concerned about similar concepts were collected into various categories and subcategories for each ques-

tion. This allows for an analysis and discussion of broader concerns, and to see what areas of potential improvement are more relevant than others. The results of this can be found in figures 5.28-5.30 below. With this, the data analysis component of the project is finished, allowing for the actual discussion of the data to take place.

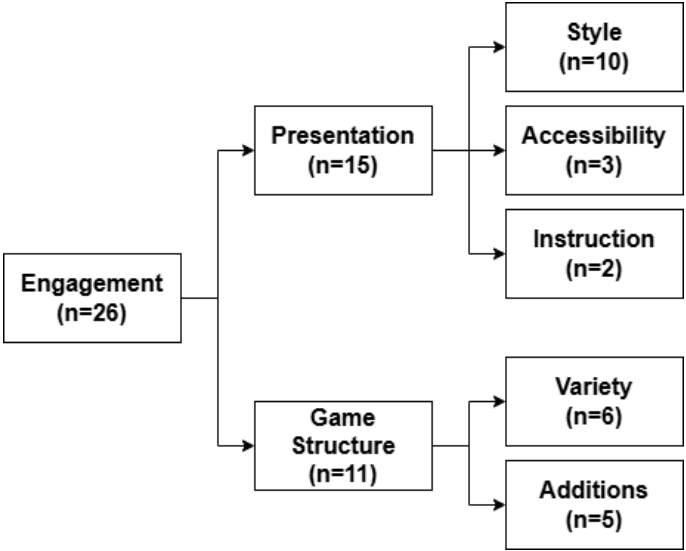


Figure 5.28: Categories of feedback for the engagement question.

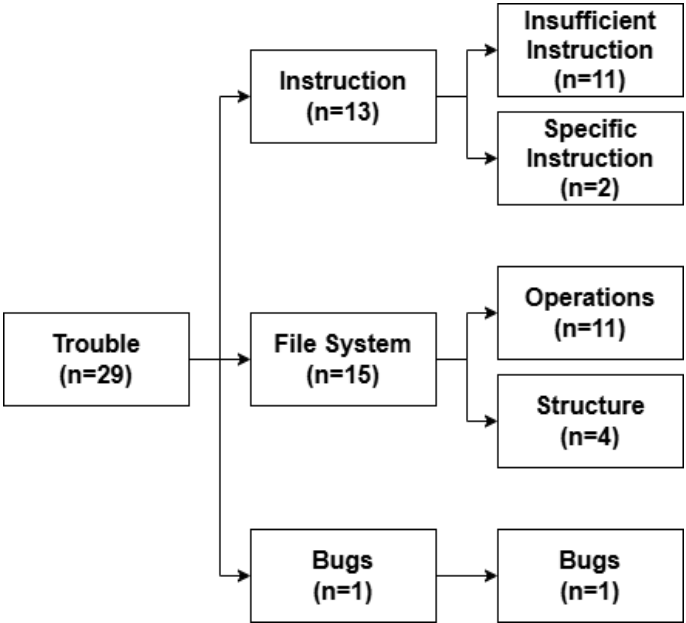


Figure 5.29: Categories of feedback for the trouble question.

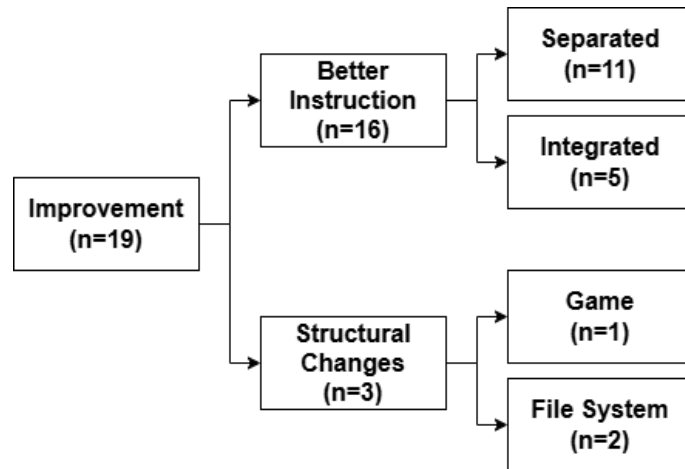


Figure 5.30: Categories of feedback for the improvement question.

CHAPTER 6

DISCUSSION & CONCLUSION

6.1 Demographic Differences

To make the discussion easier to understand, it will roughly follow the order of the analysis section. Towards that end, the first point of discussion is the demographic data. Primarily, the goal of this is to determine if any of the demographic groups did meaningfully better than any others. Reviewing the pretest scores provides a view of which demographics knew more about the digital file system from the start. From there, looking at the game performance data tells what group did the best in the actual game. Finally, reviewing the posttest data (and more importantly, the difference between it and the pretest data) tells what group is receiving the most benefit from the game. These points should aid in altering the game to not unfairly disadvantage any given demographic, and could also provide valuable insight into how the game aids students.

Overall, the demographic analysis does not meaningfully indicate that any group is unfairly disadvantaged by the game. No statistically significant difference in knowledge gain between groups was observed, although the difference between those who had played many previous educational games and those who played fewer came close. This indicates that it may be worthwhile to implement features to better reach students with less experience with games and computers. Notably, there was not a statistically significant difference in knowledge gain between male and female students, with the female students seeming to gain slightly more. This is a good result, as providing an unfair disadvantage to female students is a common concern with systems such as these.

6.2 Correlations

Moving on from the demographics, the next point of discussion is the correlations found among the different data points. This analysis was done to see if different metrics had any relation to one another. As discussed previously, the following hypotheses were selected to be checked against the apparent correlations:

1. H1: A positive correlation will exist between improvement of understanding and participation. This may indicate whether or not simply playing the game leads to knowledge gain.
2. H2: A positive correlation will exist between improvement of understanding and in-game success. This may indicate whether or not doing well in the game leads to knowledge gain.
3. H3: A positive correlation will exist between perceived usability and participation. This may indicate whether or not the game is properly engaging students.
4. H4: A positive correlation will exist between perceived usability and in-game success. This may indicate whether or not students who were more successful in-game found it to be more engaging.
5. H5: A positive correlation will exist between improvement of understanding and perceived usability. This may indicate whether or not making the game more engaging and easier to use might increase knowledge gain.

As noted previously, the number of levels encountered is used as a representation of participation, while the test improvement is used as a stand-in for knowledge gain. With these points addressed, it is possible to discuss the correlations found and what they imply.

While the lack of statistically significant correlations is insufficient to show many of the hypotheses above, some interesting results do still exist. In the overall group (seen in table 5.13), a statistically significant correlation was found between in-game performance and usability. This may suggest that making the game easier to play increases the rate at which students succeed on puzzles, or that students who had more success in the game found it to be more usable. Either conclusion is notable, as they show that improving one aspect of the game may improve others.

In the high-scoring group (table 5.15), no correlations were found to be statistically significant, suggesting that none of the hypotheses can be said to be true for this group. This, combined with the different correlations of the low-scoring group, indicates that existing knowledge is a large factor in how students interact with the game. In the low-scoring group (table 5.16), a statistically significant correlation is noted between in-game performance and knowledge gain. This is a very positive result, as it lines up with how the game is intended to teach its concepts. Overall, the correlation results suggest that the students with less initial knowledge are more affected by the game and are able to engage with it more meaningfully.

6.3 Addressing Negative Correlations

The final aspect of the correlations found in this project that needs addressed are the unexpected negative correlations found throughout the different groups. These correlations need to be addressed since they directly contradict the hypotheses that were formulated, suggesting specific ways in which the system is ineffective. While it is fortunate that none of these negative correlations are statistically significant, it is still best to address them directly. To address these correlations, it is necessary to see if some probable explanation can be given for why these came about, and why they don't necessarily imply that the system is ineffective.

Beginning with the overall group, there are two negative correlations that are present:

- Usability and levels encountered (correlation=-0.04, p-value=0.83).
- Levels encountered and test improvement (correlation=-0.00, p-value=0.96).

For both correlations, the level of correlation is rather low while the p-value is quite high. This suggests that both of these correlations can be easily attributed to random chance, not indicating that the system developed in this project is ineffective. This is especially true considering how both correlations involve the number of levels encountered, a metric that has been noted to be flawed.

Moving on to the high-scoring group, the only negative correlation that exists is between the test improvement and in-game performance. This correlation is more extreme than the negative ones found in the overall group, having a value of -0.27 and a high p-value of 0.40. When looking more closely at the data, this trend becomes explainable. In the high-scoring group, the minimum pretest score was 11 out of 12, leaving only one point to gain and many to lose. Additionally, the competency that this group had with the file system already would have led to them performing well in the game, succeeding on many more levels than they failed. These two factors (alongside random chance) could easily have combined to create this negative correlation, with students making minor mistakes on the posttest. Due to this explanation, the negative correlation found in this group is not terribly concerning.

Finally, the low-scoring group featured negative correlations in the same two pairs as the overall group. In both cases, the negative correlation was notably more extreme than that of the overall group:

- Levels encountered and usability (correlation=-0.13, p=0.59).
- Levels encountered vs improvement (correlation=-0.24, p=0.32).

Both of these negative correlations can be explained by certain trends found in the data. Looking at the data, it becomes obvious that those who had higher success rates encountered fewer puzzles, as seen in the most negative correlation of

table 5.16. This is likely due to them spending much more time on each puzzle, to the point that they were unable to encounter every puzzle due to time constraints. These slower students likely spent more time analyzing the file system of the game, gaining knowledge about how a file system should be organized. This would have allowed these more careful students to perform better on the posttest, leading to them having a higher score improvement and causing the second negative correlation. Additionally, it is likely that students who took more time per puzzle would feel that they are learning more by observing the file system structure, while those who went through more quickly may have felt less satisfied. This likely could have led to the negative correlation found between levels encountered and usability. Overall, these explanations indicate that these correlations do not suggest some inherent flaw with the game. With that discussion completed, it is time to move past the correlations and onto other topics.

6.4 Knowledge Gain T-Tests

From here, the next point of focus is the t-tests used to compare the pretests and posttest scores. Since the testing procedure had the game be played directly between the two tests, with no other activities taking place, it is reasonable to say that any significant changes in performance are caused by the game. Since the small sample size prevented the use of a control group (one being given a standard file system lecture), it won't be possible to say whether or not the game achieves better learning outcomes than a standard lesson. Nonetheless, seeing that the game provides benefits will still be an important result.

The t-test results do indicate that the overall group gained knowledge from playing the game, showing that it succeeded in its goal. Reflecting the correlation results, this gain is even greater in those with less preexisting knowledge, while those with more did not meaningfully benefit. The test performed comparing the knowledge gain of these two groups (seen in table 5.19) cements this further, showing that the low-scoring group received notably higher benefits than the high-scoring group. These points indicate that future work should focus primarily on students with less preexisting knowledge, as they are able to receive more benefits from this type of game.

6.5 Sequence Discussion

Overall, the sequence analysis informs us of a number of notable trends present in the game experience of players. First and foremost, players broadly start off confused in the very beginning of the game, before the very easy puzzles allow them to grow used to the rough expectations of the system. This behavior is expected. What is less expected is the abrupt jump in confusion when the game

moves from deletion puzzles over to creation puzzles, indicating that this transition between puzzle types is not being done smoothly enough. This confusion can be seen in the abrupt jump in mistakes and time taken around difficulty four. Many players seem to never fully recover from this confusion, being unable to finish the game at all due to spending too much time in each level. This confusion spike is also specific to this transition, not being seen anywhere else throughout the sequence data. This is notable, as some confusion would be expected when changing from the creation puzzle type to the movement type. This suggests that the current implementation of movement puzzles leaves them too similar to the creation puzzles. This does make sense, since the creation puzzles can ultimately be solved using the exact same method as the movement puzzles. Overall, these sequence results indicate that the game is largely working as intended, with two notable exceptions:

1. The creation puzzle type is not introduced smoothly enough, causing many players to be left behind.
2. The movement puzzle type is not distinct enough from the creation puzzle type. This type needs to either be made more distinct, or to be fully adopted as a subset of the creation type.

With this out of the way, it is time to move on to the final topic of discussion.

6.6 Usability Discussion

Overall, students seem to find the game decently usable, but did note certain areas of improvement. From the numerical feedback, it was clear that students felt that the game did actually improve their understanding of the digital file system to some degree. Regarding the ease of use, however, students were less satisfied and noted some more negative sentiments. This suggests that the ease with which the game can be used has notable room for improvement.

Reviewing the responses to the open-ended questions, it becomes apparent how the ease of use may be improved. One of the primary concerns that students had was that the game lacked sufficient instruction for the challenges it placed in front of them. In particular, this lack of instruction led many students to not understand how to perform the file system operations required of them, leading to difficulties. Based on this, future projects along this line should involve more integrated systems to teach students how to delete, create, and move files in the digital file system. Additionally, there were some notes given on how to make the game more engaging, such as updating the artstyle or providing more variation. Those aspects should be lower priority however, as those are both less concerning than students not knowing how to interact with the game. By addressing these usability concerns, the game could be made to more effectively teach students about the proper use and organization of the digital file system. This concludes the discussion of the data collected, allowing for the final conclusions of the project to be drawn.

6.7 Summary of Findings

To begin the conclusion, it would be appropriate to provide a recap of what was done in this project. First and foremost, a discussion occurred regarding what computer literacy is, why it is important, and how there seems to be a deficiency of it in today's society. Additionally, several other concepts were discussed and established, such as game-based learning (GBL), adaptive learning, and automatic item generation (AIG). All of these concepts were discussed not just for what they were, but also in the context of what they could provide to the project. Finally, various systems implementing these concepts were reviewed for any merits that could be brought to this project.

From there, it was established what form the project would take as well as how it would leverage GBL, adaptive difficulty, and AIG to better teach about the digital file system. The digital file system was chosen as the concept to teach due to its relevance to computer literacy and relative simplicity. Next, the implementation of the actual system was outlined. This involved the discussion of various modules (chief among them the file system, dialogue, and level modules) and how they came together to create a cohesive system. Additionally, the experiment procedures were covered, discussing how the experiment took place on two different days and what materials were used.

From there the data collected during the experiments was presented, analyzed, and discussed. This process began with the demographic analysis, where different demographic groups were compared to see if any seemed to be affected differently by the game. The demographic categories that were investigated were:

- Gender.
- Computer access at home.
- Number of educational games previously played.
- Weekly hours spent on the computer.
- Grade.
- Region.

For each demographic category, histograms were generated comparing different performance metrics. Additionally, ANOVA and t-tests were conducted on each group's knowledge gain. This analysis did not find any statistically significant results, but smaller differences were noted in the "gender" and "educational games previously played" categories. Next, certain key correlations were discussed to determine if different attributes of the game experience affected each other. Specifically, Pearson correlations were calculated not just for the overall group, but also those who scored on different sides of the median for the pretest. Overall, these correlations were not terribly revealing. Some notable results were

discovered, however, such as the radically different correlations found between the low-scoring and high-scoring groups.

T-tests were also done to compare the pretest scores of participants to the posttest scores, with the intention being to determine if the game actually provided knowledge gain to the players. The results of this test were fairly positive, with the overall group displaying statistically significant improvement. While the high-scoring group did not seem to benefit from playing the game, the low-scoring group showed a very significant benefit. The sequence data from the gameplay was then analyzed to determine how students experienced the game. Based on the analysis performed, it seems that students had a very difficult time transitioning from puzzles that tasked them with deleting a file to puzzles that asked for them to create one. Additionally, students did not seem to have difficulty transitioning from puzzles that asked for files to be created to puzzles that asked for files to be moved. Both of these results are odd and suggest that the transitions between puzzle types need to be refined.

Finally, data from a usability survey that was given to participants was analyzed and interpreted. From this, it became clear that students were broadly satisfied with the help that the game provided, although they found the game to be somewhat obtuse to play. More specifically, students noted that the game did not provide enough instruction on what was to be done, making it difficult to understand how to perform the file system operations required.

Reviewing the results that the experiment collected, there are a number of conclusions that can be drawn. Firstly, the pretest and posttest scores seem to indicate that the game does successfully teach students about the digital file system and how to effectively use it. While the lack of a control group makes it impossible to determine if this method provides better learning outcomes than a standard lecture, it is still a promising result. Additionally, it seems apparent that the game provides better learning outcomes to those with less initial knowledge of the digital file system, while those with more initial knowledge do not appear to receive any benefit. This is a fairly positive result as well, as it indicates that the game can be used as an easy means to increase the understanding of those with less initial file system knowledge.

6.8 Answering Research Questions

At the beginning of this project, a number of research questions were presented as vital topics to answer:

1. How can an educational game be made to procedurally generate puzzles about the digital file system?
2. How could this game determine what level should be generated, given the outcome of the previous level?

3. How does the proposed game improve student understanding of the digital file system?

Overall, this project has had some level of success in addressing each of these. For the first question, this project demonstrates a concrete means by which puzzles can be generated to teach about the digital file system. By observing student behavior, the system designed in this project was able to manipulate elements of the digital file system alongside in-game features to provide students with puzzles that test and improve their understanding of the digital file system.

For the second question, it is answered by the adaptive difficulty system implemented in this project. By monitoring the mistakes made by players, alongside knowledge of the difficulty of the previous puzzle, the system was able to calculate a new difficulty to be used in the puzzle the player next encountered. This then combined with the vector representation of puzzles, allowing the desired difficulty to be translated into an actual puzzle that the student could play through.

For the final question, the data collected during testing answers this. Based on the data collected, the game is not able to meaningfully improve the understanding of those players who have a high degree of existing knowledge about the digital file system. However, for those with less existing knowledge, the puzzles presented by the game seem to teach them critical aspects of how the digital file system can be organized. More specifically, the puzzles presented allowed students to become more knowledgeable of file system organization by navigating the file structures presented. Overall, this indicates that this project was a success, although not without some shortcomings that must be properly acknowledged and addressed.

6.9 Addressing Shortcomings

While it is important to acknowledge the shortcomings of a project, a better step to take is to suggest ways in which future iterations can actually address those issues. Since the shortcomings of this project seem to largely stem from the ease of use, the potential solutions here will be based on the responses gathered in the usability survey. This is not to say that the usability data will be all that these ideas are based on, as they will also involve other metrics as well.

The first changes to discuss are those that are more structural in nature. As structural changes, these ideas should be lower priority due to the relatively low number of related complaints, as well as the difficulty of their implementation. That being said, there are still certain changes to consider. One possibility is to change the nature of the puzzle solutions. More specifically, puzzle solutions could be changed to allow for various, more creative solutions. This would allow students to feel more clever and could lead to better engagement and motivation. Additionally, this could also allow for more difficult levels to be made

where more creative solutions are required, rather than simply being an alternative. Another structural change could be to better handle the transition from file creation puzzles to file movement puzzles. Currently, the two types are not terribly distinct since creation puzzles can be solved using the exact same method as movement puzzles. To make them more distinct, creation puzzles could be made to specifically expect files with no contents, preventing an existing file from simply being copied to the desired location. Alternatively, the files that students are expected to copy during movement puzzles could be removed when a creation puzzle is active. Either of these would make it such that students need to solve the two puzzle types using different methods. Overall, these are the obvious structural changes that could be made given the student feedback, and should be considered in any future projects of this sort.

An arguably easier course of action would be to update the visuals of the game to be more appealing. The game currently makes use of a pixelated art style, so it is possible that changing this to some other style would make it more appealing to players and increase engagement. Unfortunately, changing the entire art style would be fairly difficult, so it is worth investigating what other measures can be used as well. In the open-ended responses, a number of students stated that they wanted to be able to select one of several characters. Even if the only difference between these characters was visual, this could make students more invested in the game as they are now able to play as their favorite design. These options could also be extended to other aspects of the game, such as allowing students to select the font that they find easiest to read. This would be a boon to accessibility, an important part of ensuring that the game can reach any student. Overall, a number of different things could be done to overhaul the visual aspects of the game, with the ideas presented here being only a few options.

While there are improvements that can be made to other aspects of the game, the detail that seems to require the most improvement is the instruction provided to students by the game. This point is supported by the number of students who were unable to complete the game, likely due to the unusual amount of trouble encountered at the introduction of the file creation puzzles. Additionally, lack of instruction is something that many responses to the usability survey highlighted as a core issue. This is shown not only in the open-ended responses that explicitly highlight it, but also in the less than ideal "ease of use" score. Fortunately, there are a number of ways in which the instruction of the game can be improved. Firstly, the tutorial played before the actual game could be changed. Due to the time constraints of the experiment procedure, it was necessary for the tutorial to be a separate program played before the actual game. Without this constraint, it would be possible to make the tutorial be part of the main program. This change would allow for aspects of the tutorial to be integrated into the main game, providing smaller tutorials on specific concepts (such as the creation of files) as players encounter them. Finally, a hint system could be added to the game for players who are experiencing excessive trouble. These hints could provide tips for how to beat specific puzzles, while also lowering the difficulty of the next puzzle to reflect the fact that a hint was used. Overall, there are a number of ways that the

instruction of the game could be improved, although the introduction of better, more integrated tutorials is a strong starting point. Of course, addressing flaws in the existing design is not the only direction that future work can take, it would be worthwhile to discuss other directions that future work could take.

6.10 Direction of Future Work

The very final point of discussion is the possible directions that future work could take the concepts presented in this project. Since various means of addressing flaws in the system have already been reviewed, this discussion will focus on other ways of iterating on this design.

As has been mentioned at various points, the game created in this project is a proof of concept. Due to this, one of the most obvious directions for future work is to expand upon what was done here. One aspect of this may be to write a more concrete narrative for the game. Providing a more proper story could increase player engagement and motivation, especially if the story allows the player's actions to affect characters that they care about. Another possibility would be to add more systems to the game, such as branching paths or optional objectives. These would make the game more entertaining and engaging to players, which is important to GBL. These systems would likely be no more difficult to add to this project than they would be to a more standard game, due to how simple it was to integrate the digital file system into the game mechanics. Further ways that this system could aid in the classroom could also be explored, such as looking into how the game could provide information to teachers regarding what topics their students struggle with. This could take many forms, such as providing summaries of what types of puzzles students found to be the most difficult. Experiments evaluating these different ideas could also gather more responses and properly implement a control group, since the lack of a control group is a notable limitation of this study. Of course, future work could also go into an entirely new direction, taking the concepts developed in this project and applying them to other computer literacy topics.

Since the digital file system is far from the only concept related to computer literacy, it would be good to see future work explore how this game concept can be applied to other topics. Overall, games such as this allow players to grow comfortable with aspects of computer and digital literacy in a low-risk environment. This would make the application of this kind of game to other computer and digital literacy concepts very beneficial. As an example, a game could be developed where players are presented with a number of different emails. The objective of the game would be to correctly identify which emails are benign, and which emails are malicious. This would allow players to learn to identify harmful emails, without the negative consequences of incorrect identifications. Overall, this project represents a promising proof of concept for this type of game, opening the door for future research to easily expand on this work and bring the concepts presented to other aspects of computer and digital literacy.

REFERENCES

- [1] Jeanine Krath, Linda Schürmann, and Harald F.O. von Korflesch. Revealing the theoretical basis of gamification: A systematic review and analysis of theory in research on gamification, serious games and game-based learning. *Computers in Human Behavior*, 125, 2021.
- [2] Awaz Naaman Saleem, Narmin Mohammed Noori, and Fezile Ozdamli. Gamification applications in e-learning: A literature review. *Technology, Knowledge and Learning*, 27, 2022.
- [3] Renee M. Clark, Autar K. Kaw, and Rafael Braga Gomes. Adaptive learning: Helpful to the flipped classroom in the online environment of covid? *Computer Applications in Engineering Education*, 30, 2022.
- [4] Meg Coffin Murray and Jorge Pérez. Informing and performing: A study comparing adaptive learning to traditional learning. *Informing Science*, 18, 2015.
- [5] Hasan Tinmaz, Yoo Taek Lee, Mina Fanea-Ivanovici, and Hasnan Baber. A systematic review on digital literacy. *Smart Learning Environments*, 9, 2022.
- [6] Krish Chetty, Liu Qigui, Nozibele Gcora, Jaya Josie, Li Wenwei, and Chen Fang. Bridging the digital divide: Measuring digital literacy. *Economics*, 12, 2018.
- [7] Markus Appel. Are heavy users of computer games and social media more computer literate? *Computers and Education*, 59, 2012.
- [8] Maja Batez. Ict skills of university students from the faculty of sport and physical education during the covid-19 pandemic. *Sustainability (Switzerland)*, 13, 2021.
- [9] Marheny Lukitasari, Wasilatul Murtafiah, Siti Ramdiah, Rusdi Hasan, and Akhmad Sukri. Constructing digital literacy instrument and its effect on college students' learning outcomes. *International Journal of Instruction*, 15, 2022.
- [10] Branda Le, Gwendolyn A. Lawrie, and Jack T.H. Wang. Student self-perception on digital literacy in stem blended learning environments. *Journal of Science Education and Technology*, 31, 2022.
- [11] Erez Porat, Ina Blau, and Azy Barak. Measuring digital literacies: Junior high-school students' perceived competencies versus actual performance. *Computers and Education*, 126, 2018.

- [12] Angeline Kuek and Sharon Hakkennes. Healthcare staff digital literacy levels and their attitudes towards information systems. *Health Informatics Journal*, 26, 2020.
- [13] Jennifer Chipps, Loretta Le Roux, Jakobina Agabus, and Million Bimerew. Nursing informatics skills relevance and competence for final year nursing students. *Curationis*, 45, 2022.
- [14] Ester van Laar, Alexander J.A.M. van Deursen, Jan A.G.M. van Dijk, and Jos de Haan. The relation between 21st-century skills and digital skills: A systematic literature review. *Computers in Human Behavior*, 72, 2017.
- [15] Andrew M. Guess and Kevin Munger. Digital literacy and online political behavior. *Political Science Research and Methods*, 11, 2023.
- [16] Lara Bertram. Digital learning games for mathematics and computer science education: The need for preregistered rcts, standardized methodology, and advanced technology. *Frontiers in Psychology*, 11, 2020.
- [17] Liliia O. Fadieieva. Adaptive learning: a cluster-based literature review (2011-2022). *Educational Technology Quarterly*, 2023, 2023.
- [18] Kristoffer Getchell, Alan Miller, J. Ross Nicoll, Rebecca J. Sweetman, and Colin Allison. Games methodologies and immersive environments for virtual fieldwork. *IEEE Transactions on Learning Technologies*, 3, 2010. Talks about a game being used to help students who learn through practice.
- [19] Sonja Niiranen. Supporting the development of students' technological understanding in craft and technology education via the learning-by-doing approach. *International Journal of Technology and Design Education*, 31, 2021.
- [20] Danijel Dankovic, Milos Marjanovic, Nikola Mitrovic, Emilija Zivanovic, Milan Dankovic, Aneta Prijic, and Zoran Prijic. The importance of students' practical work in high schools for higher education in electronic engineering. *IEEE Transactions on Education*, 66, 2023.
- [21] Judy C.R. Tseng, Hui Chun Chu, Gwo Jen Hwang, and Chin Chung Tsai. Development of an adaptive learning system with two sources of personalization information. *Computers and Education*, 51, 2008.
- [22] Sithara H.P.W. Gamage, Jennifer R. Ayres, and Monica B. Behrend. A systematic review on trends in using moodle for teaching and learning, 2022.
- [23] Florence Martin, Yan Chen, Robert L. Moore, and Carl D. Westine. Systematic review of adaptive learning research designs, context, strategies, and technologies from 2009 to 2018. *Educational Technology Research and Development*, 68, 2020. Literature review on different adaptive learning technologies.
- [24] Nadine Pflaumer, Nancy Knorr, and Kay Berkling. Appropriation of adaptive literacy games into the german elementary school classroom. *British Journal of Educational Technology*, 52, 2021.

- [25] Synnøve Heggedal Moltudal, Rune Johan Krumsvik, and Kjetil Laurits Høydal. Adaptive learning technology in primary education: Implications for professional teacher knowledge and classroom management. *Frontiers in Education*, 7, 2022.
- [26] Mark J. Gierl, Hollis Lai, and Simon R. Turner. Using automatic item generation to create multiple-choice test items. *Medical Education*, 46, 2012.
- [27] Mark J. Gierl and Hollis Lai. Instructional topics in educational measurement (items) module: Using automated processes to generate test items. *Educational Measurement: Issues and Practice*, 32, 2013.
- [28] Peter M.C. Harrison, Tom Collins, and Daniel Müllensiefen. Applying modern psychometric techniques to melodic discrimination testing: Item response theory, computerised adaptive testing, and automatic item generation. *Scientific Reports*, 7, 2017.
- [29] Audra E. Kosh, Mary Ann Simpson, Lisa Bickel, Mark Kellogg, and Ellie Sanford-Moore. A cost-benefit analysis of automatic item generation. *Educational Measurement: Issues and Practice*, 38, 2019.
- [30] Martin Arendasy and Markus Sommer. Using psychometric technology in educational assessment: The case of a schema-based isomorphic approach to the automatic generation of quantitative reasoning items. *Learning and Individual Differences*, 17, 2007.
- [31] Martin E. Arendasy and Markus Sommer. Using automatic item generation to meet the increasing item demands of high-stakes educational and occupational assessment. *Learning and Individual Differences*, 22, 2012.
- [32] Nikolaos Pellas. Exploring interrelationships among high school students' engagement factors in introductory programming courses via a 3d multi-user serious game created in open sim. *Journal of Universal Computer Science*, 20, 2014.
- [33] Sandra Sampayo-Vargas, Chris J. Cope, Zhen He, and Graeme J. Byrne. The effectiveness of adaptive difficulty adjustments on students' motivation and learning in an educational computer game. *Computers and Education*, 69, 2013.
- [34] Steve Nebel, Maik Beege, Sascha Schneider, and Günter Daniel Rey. Competitive agents and adaptive difficulty within educational video games. *Frontiers in Education*, 5, 2020.
- [35] Fabrizio Corda, Marco Onnis, Matteo Pes, L. Davide Spano, and Riccardo Scateni. Bashdungeon: Learning unix with a video-game. *Multimedia Tools and Applications*, 78, 2019.

APPENDIX A

EXPERIMENT DOCUMENTS

A.0.1 Parent Consent Form

Research ID:

I am asking your child to participate in a research study titled "Educational Games to Teach Digital Literacy Concepts". I will describe this study to you and answer any of your questions. This study is being led by Wesley Camphouse, Computer Science Department at the New Mexico Institute of Mining and Technology. The Faculty Advisor for this study is Dr. Ramyaa, Computer Science Department at the New Mexico Institute of Mining and Technology.

What the study is about

The purpose of this research is to determine whether or not the game developed for this project achieves its goal of improving student understanding of how to use and organize the digital file system.

What we will ask you to do

I will ask your child to complete several steps in order to measure how effective the game is at teaching them about the digital file system. First, your child will be asked to complete a brief demographic survey, as well as a quiz that will help evaluate their understanding of the digital file system before the experiment. Next, your child will be asked to play through the game. After completing the game, your child will be asked to complete a quiz to measure their understanding of the digital file system after the experiment, as well as a brief questionnaire to gather their attitude towards the usability of the system. This whole process will take place inside of a class period.

Risks and discomforts

I do not anticipate any risks from participating in this research.

Benefits

By participating in this research, your child is likely to receive an improved understanding of the digital file system and how it should be utilized. This is a valuable digital literacy skill that your child is very likely to need throughout their life.

Additionally, your child's participation in this study will help improve the game, providing a tool that can be used to improve the understanding of the digital file system in other children.

Incentives for participation

No financial incentives will be provided as a result of participating in this experiment.

Privacy/Confidentiality/Data Security

Data collected during this study will not be linked directly to the identity of you or your child. Instead, data collected during this experiment will be associated with the research ID assigned to your child. The only link between your child's identity and the research results is this consent form, which will be held by the research advisor until the documents are found to be no longer useful and destroyed.

We will do our best to keep your participation in this research study confidential to the extent permitted by law; however, it is possible that other people may need to review the research records and may find out about your participation in this study. For example, the following people/groups may check and copy records about this research:

- The Office for Human Research Protections in the U. S. Department of Health and Human Services
- New Mexico Institute of Mining and Technology's Institutional Review Board (a committee that reviews and approves research studies)

Research ID:

Your child's confidentiality will be kept to the degree permitted by the technology being used. We cannot guarantee against interception of data sent via the internet by third parties.

Sharing De-identified Data Collected in this Research

De-identified data from this study may be shared with the research community at large to advance science and health. We will remove or code any personal information that could identify your child before files are shared with other researchers to ensure that, by current scientific standards and known methods, no one will be able to identify you from the information we share. Despite these measures, we cannot guarantee anonymity of your child's personal data.

Taking part is voluntary

Participation in this experiment is entirely voluntary, and participants may choose to stop participating at any point during the experiment.

If you have questions

The main researcher conducting this study is Wesley Camphouse, a graduate student at New Mexico Institute of Mining and Technology. If you have questions you may contact Wesley Camphouse at wesley.camphouse@student.nmt.edu or at (575) 706-8882.

Statement of Consent

I have read the above information, and have received answers to any questions I asked. I consent for my child to take part in the study.

Your Signature _____ Date _____

Your Name (printed) _____

Name of your child (printed) _____

A.0.2 Student Consent Form

Research ID:

Who are you/what is your research?

My name is Wesley Camphouse, and I am a student at the New Mexico Institute of Mining and Technology (otherwise known as NMT) in Socorro. I am working towards a Masters degree in computer science and engineering, and this project is going to be used as my thesis. This research involves creating and testing an educational game designed to teach late middle school/early high school students how to use and organize a digital file system. Towards this end I have created a game that generates puzzles in an attempt to adapt to player performance. The game also requires that the player uses the real file system of the computer, giving them hands-on experience.

What is the experiment?

The experiment will take place during the next session of this class, and involves a few steps. Firstly, for everyone who wishes to participate, I will collect this form along with their parental consent form. Then, each person will be assigned a research ID, which will be used in place of your name for a few quizzes and surveys. After this, all participants will take a short quiz evaluating their understanding of the digital file system. Next, all participants will be allowed to play the actual game for roughly 20 minutes. At the end of that time, participants will be given another short quiz as well as two brief surveys. One survey will collect some basic demographic information, and the other will give you a chance to share your thoughts and feelings regarding the game that you played.

Data Collection

Due to the nature of the experiment, some data will be collected from you. It is important to note that all data collected will **only be associated with your research ID**. This means that the data collected during this experiment will not be able to be traced back to you.

Firstly, both of the quizzes will collect information on your understanding of the digital file system. Secondly, the demographic survey will collect certain pieces of relevant demographic information, such as how many computers you have at home and how much time you spend on them. Thirdly, the sentiment survey will collect any thoughts you feel like sharing regarding the game that you played. Finally, the game itself will collect some performance data as you play, such as how much time you spend in levels and how many points you earn.

Consent

Participation in this experiment is completely voluntary; if you do not wish to participate, simply do not sign or turn in this form. If you do not turn in both this form and the parental consent form with signatures, you will not be able to participate in the experiment.

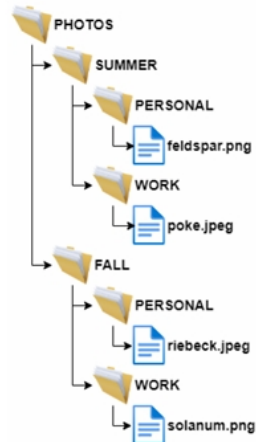
Your name (printed): _____

Your signature: _____ Date: _____

A.0.3 Pretest

Research ID: _____

1. For parts a, b, and c, consider the following file structure (a name in all-caps indicates a folder, while a lowercase name indicates a file):



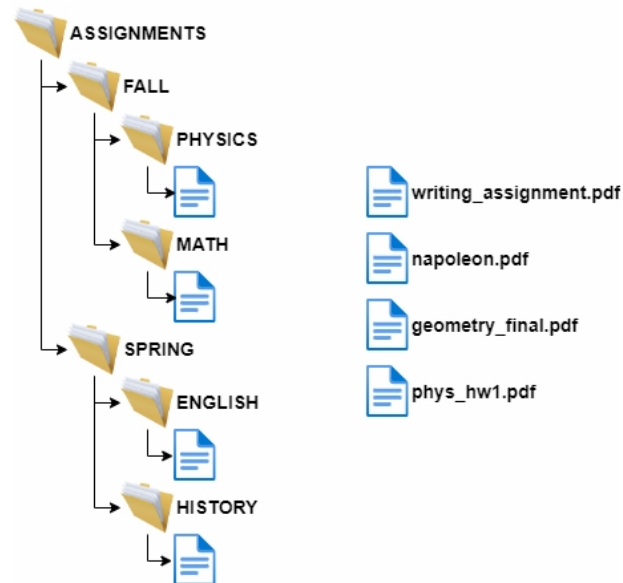
- a. What series of movements would bring you from the *PERSONAL* folder (the folder containing *feldspar.png*) into the folder containing *poke.jpeg*?
- Enter PHOTOS, enter SUMMER, enter WORK.
 - Exit the current folder, enter WORK.
 - Enter SUMMER, enter WORK.
 - I'm already in the folder containing *poke.jpeg*.
- b. If you are in the first *WORK* folder (the folder containing *poke.jpeg*), how many times will you have to exit folders when navigating to *solanum.png*?
- 1
 - 2
 - 3
 - 4

2. For parts a, b, and c, consider the following file structure (a name in all-caps indicates a folder, while a lowercase name indicates a file):

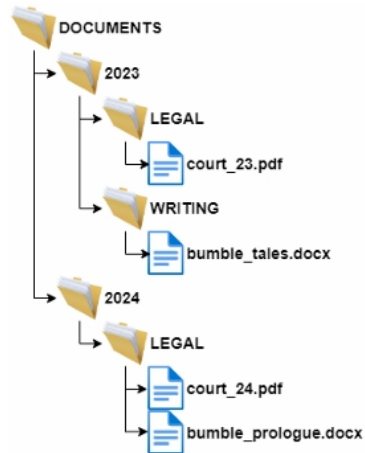


- a. One of the files is in the wrong location. Which file is it?
- lego_star_wars.exe
 - peggle_capture.png
 - overlord.exe
 - peggle.exe
- b. Where should that file be moved to in order to properly organize the file system?
- GAMES
 - APPS
 - SCREENSHOTS
- c. In what folder should you put the file starwars_screenshot.png?
- SCREENSHOTS
 - APPS
 - GAMES

3. On the left side of the image below, a file structure with several folders and slots for files is presented. On the right side is a list of various files. Arrange the files by drawing a line between the file on the right and the correct location on the left. Do this so that you create a properly organized file system. (Note that a name in all-caps indicates a folder, while a lowercase name indicates a file.)



4. For parts a, b, and c, consider the following file structure (a name in all-caps indicates a folder, while a lowercase name indicates a file):

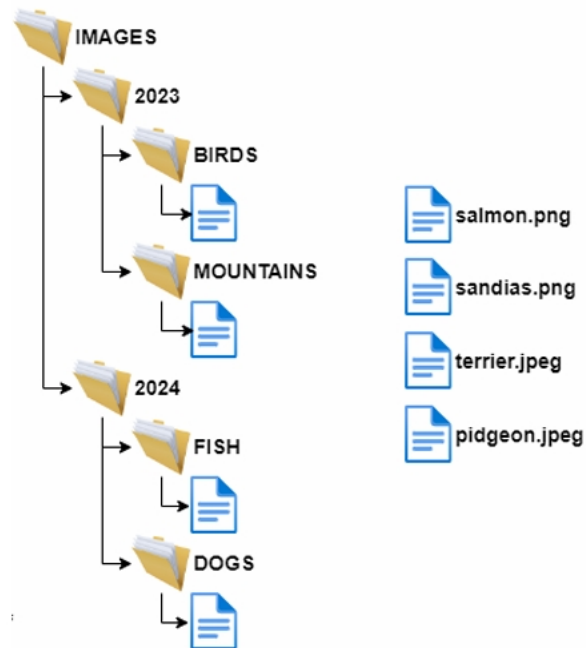


- a. One of the files in the file structure is not in an appropriate location. Circle the file that is disorganized.
- b. There is currently no folder in the file system that would be a good place to store that file. Provide the name of a new folder that would be a good place to store the file.
- WRITING
 - LEGAL
 - 2025
 - PICTURES
- c. Indicate the location that your new folder should be placed in.
- DOCUMENTS/2023
 - DOCUMENTS/2024
 - DOCUMENTS/2023/WRITING
 - DOCUMENTS/2023/LEGAL

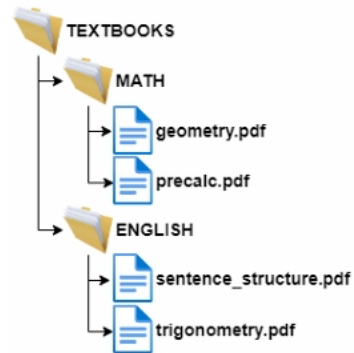
A.0.4 Posttest

Research ID: _____

1. On the left side of the image below, a file structure with several folders and slots for files is presented. On the right side is a list of various files. Arrange the files by drawing a line between the file on the right and the correct location on the left. Do this so that you create a properly organized file system. (Note that a name in all-caps indicates a folder, while a lowercase name indicates a file.)

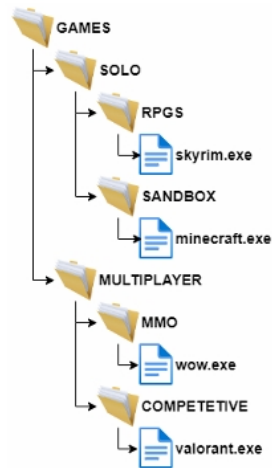


2. For parts a, b, and c, consider the following file structure (a name in all-caps indicates a folder, while a lowercase name indicates a file):



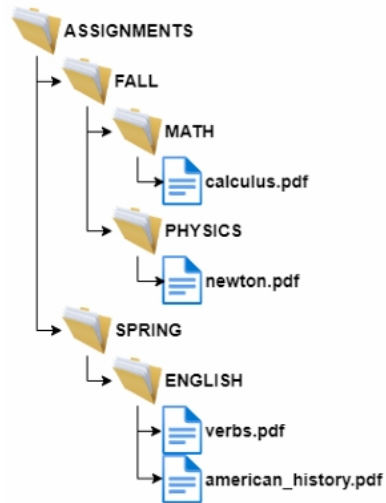
- a. One of the files is in the wrong location. Which file is it?
- geometry.pdf
 - precalc.pdf
 - sentence_structure.pdf
 - trigonometry.pdf
- b. Where should that file be moved to in order to properly organize the file system?
- TEXTBOOKS
 - MATH
 - ENGLISH
- c. In what folder should you put the file nouns.pdf?
- TEXTBOOKS
 - MATH
 - ENGLISH

3. For parts a, b, and c, consider the following file structure (a name in all-caps indicates a folder, while a lowercase name indicates a file):



- a. What series of movements would bring you from the *RPGS* folder (the folder containing *skyrim.exe*) into the folder containing *minecraft.exe*?
- Enter GAMES, enter SOLO, enter SANDBOX.
 - Exit the current folder, enter SANDBOX.
 - Enter SOLO, enter SANDBOX.
 - I'm already in the folder containing *minecraft.exe*.
- b. If you are in the *SANDBOX* folder (the folder containing *minecraft.exe*), how many times will you have to exit folders when navigating to *valorant.exe*?
- 1
 - 2
 - 3

4. For parts a, b, and c, consider the following file structure (a name in all-caps indicates a folder, while a lowercase name indicates a file):



- a. One of the files in the file structure is not in an appropriate location. Circle the file that is disorganized.
- b. There is currently no folder in the file system that would be a good place to store that file. Provide the name of a new folder that would be a good place to store the file.
 - i. MATH
 - ii. PHYSICS
 - iii. WINTER
 - iv. HISTORY
- c. Indicate the location that your new folder should be placed in.
 - i. ASSIGNMENTS/FALL
 - ii. ASSIGNMENTS/SPRING
 - iii. ASSIGNMENTS/FALL/MATH
 - iv. ASSIGNMENTS/FALL/PHYSICS

A.0.5 Demographic Survey

Research ID: _____

This is a brief survey meant to collect some basic demographic information. For each question, circle the answer that is the most accurate. While your answers will be tied to your research ID, it will not be possible to link them back to your identity.

1. What grade are you in?

- | | |
|--------|---------|
| a. 7th | b. 8th |
| c. 9th | d. 10th |

2. What is your gender?

- | | |
|----------------------|-----------|
| a. Male | b. Female |
| c. Prefer not to say | |

3. How many hours do you use a computer per week? (Count all time spent using a computer, including using a computer at school)

- | | |
|---------------|----------|
| a. 0-2 | b. 2-5 |
| c. 5-15 | d. 15-30 |
| e. 30 or more | |

4. How many hours per week do you use a computer for non-recreation (such as schoolwork)? (Count all time spent using a computer, including using a computer at school)

- | | |
|---------------|----------|
| a. 0-2 | b. 2-5 |
| c. 5-15 | d. 15-30 |
| e. 30 or more | |

5. How many hours per week do you spend playing digital games? (This includes games on any electronic device, such as a cell phone or computer)

- a. 0-2
- b. 2-5
- c. 5-15
- d. 15-30
- e. 30 or more

6. How many educational games (such as typing games or games on mathisfun.com) have you played before?

- a. 0-1
- b. 2-3
- c. 4-5
- d. 6 or more

7. What subject have you used the most educational games for? (Leave this question blank if you have never played any educational games before the one used today)

8. Do you have access to a computer at home? (Circle no if there is a computer in your home, but you do not have permission to use it)

- a. Yes
- b. No

A.0.6 Usability Survey

Research ID #: _____

#	Statement/Question	Highly Agree	Agree	Neutral	Disagree	Highly Disagree
1	Playing this game has improved my understanding of digital file system navigation.					
2	I feel more confident navigating from one location to another in the digital file system after playing the game.					
3	After playing the game I feel better about my ability to navigate to a location in the digital file system.					
4	Playing this game has improved my understanding of how to organize a digital file system.					
5	I feel more confident in my ability to effectively organize a digital file system after playing this game.					
6	After playing the game I feel better about my ability to organize a digital file system.					
7	Playing the game has improved my ability to locate files in a well-organized digital file system.					
8	I feel more confident in my ability to locate files in a well-organized file system.					
9	After playing the game I feel better about my ability to locate files in a well-organized file system.					
10	Playing the game has improved my understanding of the various operations in a digital file system.					
11	I feel more confident in my understanding of the various operations in a digital file system.					
12	After playing the game I feel better about my understanding of the various operations in a digital file system.					

Research ID #: _____

#	Statement/Question	Highly Agree	Agree	Neutral	Disagree	Highly Disagree
13	I found the controls of the game to be intuitive and easy-to-use.					
14	The controls of the game were presented in a way that allowed me to use them effectively.					
15	I understood the controls and they did not stand in the way of me completing the game.					
16	I found the goals of each level to be well-communicated and understandable.					
17	The goal of each level was presented in a way that helped me understand what I was supposed to do.					
18	I understood the goal of each level and the explanation of these goals did not stand in the way of me completing the game.					
19	I found the incorporation of the digital file system into the game to be intuitive.					
20	The incorporation of the digital file system into the game was intuitive and helped me understand how to use it.					
21	I understood how the digital file system was being incorporated into the game and it did not stand in the way of me completing the game.					

Research ID #: _____

Describe any ways that the game could be made more engaging.

Describe any memorable difficulties you had understanding the game.

Describe any suggestions you have on how to make the game better communicate the skills involved in using the file system.

DESIGN AND EVALUATION OF
AN EDUCATIONAL GAME TO
TEACH FILE SYSTEM MANAGEMENT

by

Wesley Camphouse

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the last page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and may require a fee.

